



Technische Universität München
Fakultät für Mathematik

Partikelbasierte Fluidsimulation für interaktive Anwendungen

Diplomarbeit von Leo Wandersleb

Aufgabensteller: Prof. Dr. Thomas Huckle
Betreuer: Prof. Dr. Rüdiger Westermann
Abgabetermin: 17. Januar 2005

Hiermit erkläre ich, die Diplomarbeit selbständig und nur mit den angegebenen Hilfsmitteln angefertigt zu haben.

München, den 16. Januar 2005

(Leo Wandersleb)

Zusammenfassung

Wasser zu simulieren ist im Bereich der Computergrafik immer noch eine große Herausforderung. Auch wenn auf dem Gebiet der Offline-Simulation zunehmend plausible Resultate zu sehen sind, wie z.B. in den Kinofilmen „Titanic“ (1997), „Antz“ (1998) oder „Finding Nemo“ (2003), ist das Simulieren von Flüssigkeiten im interaktiven Bereich ein weitgehend unbestelltes Feld. Dank der erreichten Leistungsstärke aktueller PCs ist eine solche Simulation mit vernünftigem Aufwand realisierbar.

In der vorliegenden Arbeit soll der Ansatz von Matthias Müller, David Charypar & Markus Gross [2] fortgeführt werden und durch eine neuartige Datenstruktur der Speicherbedarf bei gleichzeitiger Vergrößerung des Simulationsgebiets verringert werden. Während das Programm zur vorliegenden Arbeit bei höheren Bildraten mit unter 15MB RAM auskommt, konnte das Simulationsgebiet auf das etwa hunderttausendfache gesteigert werden.

Inhaltsverzeichnis

1	Einführung	3
1.1	Zweck und Aufbau dieser Diplomarbeit	4
2	Physikalische Grundlagen	5
2.1	NAVIER-STOKES-Gleichungen	5
2.2	Smoothed Particle Hydrodynamics	7
3	Kollisionsdetektion	9
3.1	Explizite Raumpartitionierung	9
3.2	Implizite Raumpartitionierung	11
3.2.1	Flüssigkeitspartikel	14
3.2.2	Kontrollpartikel	14
4	Visualisierung	16
4.1	Der Marching Cube Algorithmus	17
4.2	A grid to march on	19
4.2.1	Ecken bestimmen	19
4.2.2	Gitterzellen zu den Ecken	19
4.2.3	Erzeugung der Oberflächenpunkte	21
4.2.4	Verbinden der Oberflächenpunkte	22
4.3	Deformiertes Gitter	22
4.4	Flächen höherer Ordnung	24
4.5	Verfeinerung des Gitters	29
5	Anwendung	30
5.1	Systemvoraussetzungen	30
5.2	Benötigte Dateien	31
5.3	Notation	31
5.4	Public member von <code>Fluid</code>	32
5.4.1	Konstruktor	32
5.4.2	<code>trianglelist2boundary</code>	33
5.4.3	<code>trianglelist2teleport</code>	33
5.4.4	<code>trianglelist2shift</code>	34
5.4.5	<code>trianglelist2setspeed</code>	35

5.4.6	tetraederlist2liquid	35
5.4.7	progress	36
5.4.8	movingparticlecount	36
5.4.9	boundaryparticlecount	36
5.4.10	particlecount	37
5.4.11	get_particlearray	37
5.4.12	get_surfacegrid	37
5.4.13	set_height_function	38
5.4.14	set_height_function_normal	39
5.4.15	height_function	39
5.4.16	height_function_normal	39
5.4.17	minxyz	39
5.4.18	maxxyz	40
5.4.19	particleradius	40
5.5	Die Beispielimplementierung	40
5.5.1	Programmstart	40
5.5.2	Tastaturbelegung	40
6	Resultate	42
7	Ausblick	46
8	Danksagung	47
	Literaturverzeichnis	48

Kapitel 1

Einführung

Obwohl schon viel gearbeitet und geforscht wurde auf dem Gebiet der Fluidsimulation, steckt sie immer noch in den Kinderschuhen. Insbesondere gibt es noch keine interaktiven Verfahren, die auf großen Gebieten Wasser im Raum, also nicht als Höhenfeld realisieren. Anwendungen hierfür wären etwa Wasserfälle, Getränke oder Wasserwerfer in Computerspielen, aber auch zum Beispiel Blut in Operationssimulatoren.

Den Anstoß zu einem um Größenordnungen umfangreicheren Simulationsgebiet brachte Herr Prof. Dr. Westermann, als er vorschlug, Zellinformationen nur noch beim Partikel zu speichern und nach diesem Zellindex die Partikel zu sortieren. Durch diese implizite Anordnung der Zellen wird erreicht, dass der Speicherbedarf unabhängig von der Größe des simulierten Gebiets wird und meine Software keine 15MB benötigt, um 3000 Partikel in einem Gebiet bestehend aus $(100.000)^3$ Zellen zu simulieren. Möchte man 500.000 Partikel verwenden, so stehen immernoch $(16.000)^3$ Zellen zur Verfügung.

Laut Nick Fosters & Ronald Fedkiws „*Practical animation of liquids*“ [1] wurde z.B. für die Simulation der Schlammdusche von Dreamworks „Shrek“ lediglich ein Gitter von $150 \times 200 \times 150$ Zellen verwendet. Um auf meiner Datenstruktur Physik berechnen zu können und vor allem um die zugehörige Oberfläche zu konstruieren, sind allerdings sehr spezialisierte Ansätze gefragt. So orientiert sich diese Arbeit im Groben an [2] von Matthias Müller, David Charypar & Markus Gross, passt deren Ansätze aber auf die erwähnte Datenstruktur an. Außerdem wird die auf ATI-Grafikkarten vorhandene TRUFORM™-Erweiterung genutzt, um auf einem relativ groben Gitter mittels des Marching Cube Algorithmus erzeugte, durch ebene Teilflächen charakterisierte, also eckige, Oberflächen rund darzustellen.

1.1 Zweck und Aufbau dieser Diplomarbeit

Während ursprünglich geplant war, aus den Ergebnissen aus [2] und [6] eine interaktive Simulation für Flüssigkeiten mit verschiedenen Eigenschaften zu entwickeln, entstanden aus der „entdeckten“ Methode zur Kollisionsdetektion ganz neue Anforderungen und die Physik trat zunächst in den Hintergrund. Außerdem hat diese Arbeit möglicherweise auch Relevanz für astronomische Simulationen, wofür die Methoden der Smoothed Particle Hydrodynamics (SPH) ursprünglich entwickelt wurden, da hier Verfahren aufgezeigt werden, die den Simulationsbereich herkömmlicher Anwendungen um Größenordnungen übersteigen.

So werden in Kapitel 2 die physikalischen Grundlagen erarbeitet, sowie die Smoothed Particle Hydrodynamic als Methode zur numerischen Lösung von Flussproblemen vorgestellt. In Kapitel 3 wird auf die für SPH benötigte Kollisionsdetektion eingegangen und ein neues Verfahren eingeführt. In Kapitel 4 werden Methoden zur schnellen Oberflächenerzeugung entwickelt. Kapitel 5 stellt ein kleines Demonstrationsprogramm vor, welches sich auch auf der beigefügten CD befindet. In Kapitel 6 sind einige Screenshots, so wie Angaben zur Performance zu finden. Kapitel 7 gibt schließlich einen Ausblick auf Anwendungsmöglichkeiten und Entwicklungspotential.

Kapitel 2

Physikalische Grundlagen

Grundsätzlich gibt es zwei Ansätze, Physik von Wasser zu berechnen.

- Auf einem festen Gitter, was sich empfiehlt, wenn im Wesentlichen alle Gitterpunkte innerhalb des simulierten Mediums sind, also zum Ergebnis beitragen.
- Auf Punkten, die sich mit dem Geschwindigkeitsfeld mitbewegen (LAGRANGE'scher Formalismus), was vorzuziehen ist, wenn die Verteilung des Mediums im Raum nicht a priori bekannt ist bzw. sehr stark variiert (Spritzeffekte, Regen, ...).

Doch bevor hierauf näher eingegangen wird, werden Gleichungen benötigt, die Fluiddynamik beschreiben.

2.1 NAVIER-STOKES-Gleichungen

Schon Anfang des 19ten Jahrhunderts entwickelten der Franzose Navier und der Brite Stokes ein Gleichungssystem, um das Verhalten von Flüssigkeiten zu beschreiben. Dabei werden Viskosität, Druck, Körperkräfte sowie Massen- und Impulserhaltung berücksichtigt.

Die Viskositätskraft

$$\frac{\vec{F}_{\text{Viskosität}}}{V} = \eta \nabla^2 \vec{v},$$

die proportional zur Viskosität η in Richtung der Geschwindigkeitsänderung wirkt, die Druckkraft

$$\frac{\vec{F}_{\text{Druck}}}{V} = -\nabla P,$$

die entgegen dem Druckgradienten, also in Richtung des größten Druckabfalls wirkt, und eine sogenannte Körperkraft

$$\frac{\vec{F}_{\text{Körper}}}{V} = F,$$

wie zum Beispiel die Gravitationskraft ergeben mit dem Impuls $\vec{p}$ und

$$\frac{\vec{F}}{V} = \frac{1}{V} \frac{d\vec{p}}{dt} = \rho \frac{D\vec{v}}{Dt} = \rho \frac{\partial \vec{v}}{\partial t} + \rho \vec{v} \cdot \nabla \vec{v}$$

(Newtons Gesetz der Fluidodynamik) die Gleichung

$$\rho \frac{\partial \vec{v}}{\partial t} + \rho \vec{v} \cdot \nabla \vec{v} = F - \nabla p + \eta \nabla^2 \vec{v}.$$

Teilt man nun durch ρ , so erhält man

$$\frac{\partial \vec{v}}{\partial t} + \vec{v} \cdot \nabla \vec{v} = \frac{F}{\rho} - \frac{\nabla p}{\rho} + \nu \nabla^2 \vec{v}, \quad (2.1)$$

wobei $\nu = \frac{\eta}{\rho}$ als kinematische Viskosität bezeichnet wird.

Gleichung 2.1 und die Kontinuitätsgleichung

$$\frac{\partial \rho}{\partial t} = -\nabla(\rho \cdot \vec{v}) = -\vec{v} \cdot \underbrace{\nabla \rho}_{=0} - \rho \nabla \cdot \vec{v} = -\rho \nabla \cdot \vec{v} \quad (2.2)$$

werden zusammen als NAVIER-STOKES-Gleichungen bezeichnet.

Die Kontinuitätsgleichung unter Annahme der Inkompressibilität (Gleichung 2.2) beschreibt in Worten, dass die Änderung der Dichte proportional zum Negativen der Divergenz des Geschwindigkeitsfeldes, also zum Zufluss ist. Ein positiver Zufluss bedeutet ein Steigen der Dichte.

Diese Einführung zu den NAVIER-STOKES Gleichungen und weiterführende Links finden sich bei [5].

Die vorliegende Arbeit verwendet den LAGRANGE'schen Formalismus, wodurch sich, wie in [2] beschrieben, einige Vereinfachungen ergeben. So kann die für den Massenerhalt zuständige Gleichung 2.2 ganz weggelassen werden, da die Masse auf Partikeln gespeichert ohnehin konstant bleibt.

Da sich die Partikel mit der Flüssigkeit bewegen, wird auch der konvektive Term aus Gleichung 2.1 nicht benötigt, und die linke Seite von Gleichung 2.1 ist nur noch die Zeitableitung der Geschwindigkeit $d\vec{v}/dt$. Mit der Kraft

$$f = F - \nabla p + \nu \nabla^2 \vec{v} \quad (2.3)$$

ergibt sich als Beschleunigung $\vec{a}$ für das i -te Partikel

$$\vec{a}_i = \frac{d\vec{v}_i}{dt} = \frac{f_i}{\rho_i}. \quad (2.4)$$

Um diese Gleichungen aber numerisch zu lösen, also insbesondere die physikalischen Größen im Raum zu bestimmen, wird hier Smoothed Particle Hydrodynamics (SPH) verwendet.

2.2 Smoothed Particle Hydrodynamics

Im SPH-Formalismus wird die Verteilung einer physikalischen Größe u im Raum durch Glätten der auf Partikelpositionen gespeicherten Werte $\langle u \rangle$ approximiert.

$$\langle u(\vec{x}) \rangle = \int_V u(\vec{x}) W(\vec{x} - \vec{x}', h) dV$$

Dabei muss der Glättungskern $W(\vec{x} - \vec{x}', h)$ normiert sein, also

$$\int_V W(\vec{x} - \vec{x}', h) dV = 1 \quad (2.5)$$

gelten, und unter Verwendung einer Glättungslänge $h \rightarrow 0$ muss der geglättete Wert $\langle u(\vec{x}) \rangle$ gegen $u(\vec{x})$ gehen.

$$h \rightarrow 0 \Rightarrow \langle u(\vec{x}) \rangle \rightarrow u(\vec{x}) \quad (2.6)$$

Notation: In späteren Kapiteln ist von *Partikelradius*, *Interaktionsradius* oder kurz *Radius* die Rede. Damit ist immer die dann für alle Partikel und Glättungskerne konstante *Glättungslänge* gemeint.

Mit Hilfe dieser Glättungskerne kann nun u an jedem Punkt im Raum als Summe über alle geglätteten Partikel bestimmt werden. Dieser Wert ist im Allgemeinen nicht exakt, aber Olaf Kessel-Deynet zeigt in [7], dass es schon genügt, zusätzlich zu den Gleichungen 2.5 und 2.6 zu verlangen, dass W gerade ($W(\vec{x}, h) = W(-\vec{x}, h)$) ist, um eine in h quadratische Fehlerordnung zu erhalten.

$$\langle u(\vec{x}) \rangle = \sum_i \frac{m_i}{\rho(\vec{x}_i)} u(\vec{x}_i) W(|\vec{x} - \vec{x}_i|, h),$$

wobei m_i , $\rho(\vec{x}_i)$ und $u(\vec{x}_i)$ die entsprechenden Größen am i -ten SPH-Partikel sind.

SPH ist durch die Möglichkeit der freien Platzierung von Partikeln im Simulationsgebiet speziell geeignet, Prozesse, die sich auf einem komplizierten räumlichen Gebiet abspielen, zu simulieren, da die Partikel zum Zweck erhöhter Genauigkeit in relevanten Bereichen lokal dichter und in anderen, die nur geringen oder gar keinen Einfluss auf die Simulation haben, auch gar nicht platziert werden können. Auch ist eine Neuplatzierung dieser Partikel für jeden Zeitschritt meist sinnvoll. So eben auch bei der Simulation von Wasser. In meiner Arbeit transportiere ich die Partikelposition schlicht mit der Geschwindigkeit des Wassers. Andere Ansätze, die dies lediglich als erste Approximation wählen, werden zum Beispiel in [6] verwendet.

In dieser Arbeit wird das Gleichungssystem 2.4 gelöst, indem in einem ersten Durchlauf für jedes Partikel die Dichte ρ als Summe der geglätteten Dichten aller Partikel bestimmt wird und in einem zweiten Durchlauf die von ρ abhängigen geglätteten Kräfte aufsummiert werden. Da die Glättungslänge h klein ist, muss nicht über alle Partikel summiert werden, sondern nur über Partikel in der jeweiligen h -Umgebung. Um das zu optimieren, wird eine effiziente Kollisionsdetektion benötigt.

Kapitel 3

Kollisionsdetektion

Da der Aufwand für die n -Partikel-Kollisionsdetektion die Ordnung n^2 hätte (jedes Partikel kommt für jedes andere Partikel als Kollisionspartner in Frage), ist es notwendig, sich hierüber Gedanken zu machen. Um ein plausibles Ergebnis der Flüssigkeitsbewegung zu erhalten, genügt es, wie in 2.2 beschrieben, die Physik in jedem Punkt der LAGRANGE'schen Simulation (Partikel) nur auf einem kompakten Träger zu berechnen. Der Radius dieses Trägers bestimmt maßgeblich die Komplexität der Simulation. Da die Partikel keinen Einfluß auf das gesamte Gebiet haben, sondern nur auf ihre unmittelbare Umgebung, empfiehlt sich nun eine Raumpartitionierung. Es wird nur der Abstand zu den Partikeln ermittelt, deren Raumpartition sich mit dem Träger der physikalischen Größen des betrachteten Partikels schneidet. Dadurch wird der Aufwand von n^2 auf $\sum_{i=1}^n k_i^2 \leq k * n$ reduziert, wobei k_i die Anzahl der Partikel in der Partition des i -ten Partikels und k die maximale Anzahl an Partikeln pro Raumpartition ist. Nimmt man den Bereich B_r jedes Partikels als Raumpartition, so erhält man das von der Physik abhängige optimale k^{opt} . Damit ist k ein Maß dafür, zu wie vielen Partikeln der Abstand berechnet werden muss. Also gibt k/k^{opt} an, wie viele Abstände zu viel gemessen werden, was insbesondere dann eine große Rolle spielt, wenn die Physik bei einer Kollision sehr billig zu berechnen, die Ermittlung des Abstandes jedoch schon vergleichbar teuer ist.

3.1 Explizite Raumpartitionierung

In einem ersten Schritt erprobte ich die Kollisionsdetektion mittels expliziter Speicherung der Partikelindizes pro Raumpartition mit einem regelmäßigen Gitter als Raumpartitionierung. Also etwa 200^3 Zellen mit der Möglichkeit, pro Zelle 20 Partikelindizes zu speichern. Dies setzt voraus, dass die Physik niemals mehr als 20 Partikel pro Zelle zulässt. Unter Verwendung von vier

Byte Adressen für die Partikel resultierte das in einem Speicherbedarf zu Programmstart von $200^3 * 20 * 4B = 640MB$.

Der auf diesem Gitter arbeitende Algorithmus musste entweder berechnen, mit welcher der benachbarten 26 Zellen das Partikel interagieren kann ($20 \leq k \leq 8 * 20 = 160$), oder man durchsucht pauschal alle 27 Zellen, die das Partikel enthalten bzw. die das Partikel enthaltende Zelle umgeben ($k = 27 * 20 = 540$). Zweiteres ist wohl das Praktikablere, da hier mit einer Zellkantenlänge $\approx r$ der Wert k sehr nahe k^{opt} ist. Ersteres macht nur Sinn, wenn die Zellkantenlänge $\ll r$ ist, was wiederum bedeuten würde, dass mit 20 Partikeln pro Zelle nur wenige dieser Partikel interagieren können.

Eine dynamische Speicherbelegung reduzierte zwar den Speicherbedarf auf 1B pro Zelle für leere Zellen, was mir allerdings zum einen nicht die schon angekündigten mehr als 10.000^3 Zellen ($\hat{=} 10.000^3 B = 1TB$) ermöglicht, zum anderen für manche Situationen, wie etwa Regen, den Aufwand für *Speicher freigeben* und *Speicher reservieren* extrem in die Höhe treibt.

Exkurs (Jedem Partikel seine Zelle): Um nur die Partikel einer Zelle auf Abstand $d < r_0$ prüfen zu müssen, können auch verschobene reguläre Gitter der Kantenlänge $4r_0$ erzeugt werden.

Eindimensional benötigt man zwei Gitter:

- Gitter $g = 0$: $\{[j(4r_0); (j+1)(4r_0)] | j \in \mathbb{N}\}$
- Gitter $g = 1$: $\{[(j + \frac{1}{2})(4r_0); (j + \frac{3}{2})(4r_0)] | j \in \mathbb{N}\}$

Als Kollisionspartner für ein Partikel an Position x kommen nur Partikel in Zelle (g, j) mit

$$\begin{aligned} g &= \left\lfloor \left(\frac{x}{4r_0} - \left\lfloor \frac{x}{4r_0} \right\rfloor \right) * 4 - 2 \right\rfloor \\ j &= \frac{x - 2gr_0}{4r_0} \end{aligned}$$

in Frage.

Im Zweidimensionalen werden vier und im Dreidimensionalen gar 8 Gitter benötigt. Zur Veranschaulichung des zweidimensionalen Falls dient Abb. 3.2 auf Seite 13.

Mir ist keine Implementierungen dieser vielen Gitter bekannt, wohl auch, da ein begrenzender Faktor der Speicherplatz ist, der bei einer expliziten Speicherung aller Zellen im Simulationsgebiet eine Verachtfachung ($640MB * 8 = 5,12GB$) des Speicherbedarfs bedeutet. Deshalb wird in den angrenzenden Zellen in *einem* festen Gitter gesucht.

3.2 Implizite Raumpartitionierung

Um diesen gigantischen Speicherbedarf zu vermeiden, schlug Herr Prof. Dr. Rüdiger Westermann vor, zwar diesen Weg der vielen Gitter zu gehen, die Gitterinformation aber nur bei den Partikeln zu speichern, diese zu sortieren und dadurch in einer Zelle befindliche Partikel zu identifizieren. Bekannte Sortieralgorithmen haben zwar bei n Elementen eine Laufzeitordnung von $n \log n$. Das ist aber gerade im interaktiven Bereich auf Grund der hohen Kosten für die Physik, die wie in 3 gezeigt eine Ordnung von kn haben, vernachlässigbar. Beispielläufe zeigen, daß selbst bei 1 Mio. Partikeln keine 10% der Rechenzeit für Sortieren verwendet werden. In 3.1 habe ich zwar behauptet, man bräuchte im Dreidimensionalen 8 Gittersysteme um garantieren zu können, dass sich alle Kollisionspartner eines Partikels in einer wie oben bestimmten Zelle befinden. Was aber unter 3.1 nur eine theoretische Überlegung war, ist bei der impliziten Zelleinteilung Notwendigkeit. In einer Sortierung des $\mathbf{R}^3$ kann es nur in zwei Richtungen Nachbarn geben. Die Zellen in der nächsten Ebene liegen in der sortierten Liste in unbekannter Ferne. Wenn aber diese zwei Richtungen, in denen man die Nachbarzelle finden kann, ausgenutzt werden, so braucht man im $\mathbf{R}^3$ nur noch vier Listen.

Im Eindimensionalen wird sofort klar, dass man kein Gitter braucht. Man muss die nach der Koordinate sortierte Liste vom betrachteten Partikel aus nur so lange in beide Richtungen durchlaufen, bis man auf ein Partikel trifft, das zu weit weg ist, um zu kollidieren. Alle Weiteren in dieser Richtung können nur noch weiter weg sein.

Um im Höherdimensionalen sortieren zu können, wird ein etwas komplizierteres Kriterium benötigt. Hier ist dies ein Zellindex, der sich bei mir wie folgt zusammensetzt:

Die höchsten Bits repräsentieren die Position in z -Richtung, danach kommt die Position in y -Richtung und danach die in x -Richtung jeweils als Vielfaches der gewählten Zellgröße (deren Kantenlänge), die in meiner Software mit $4r$ minimal gewählt wurde.

Abb. 3.1 zeigt in 2D, welches Simulationsgebiet bei der Wahl des grün schraffierten Gebietes und der grün schraffierten Partikelgröße zur Verfügung steht. Aus der Bit-weisen Einteilung des Zellindex ergibt sich, dass das Simulationsgebiet in jeder Raumdimension separat auf die Zweierpotenz $*4r$ vergrößert wird, die größer oder gleich der vorgegebenen Abmessung ist.

Exkurs: Da der Zellindex modulo der über die Größe des Simulationsgebiets gewählten Zellanzahl in der jeweiligen Richtung ermittelt wird, werden Partikeln, die sich außerhalb des eigentlichen Simulationsgebiets bewegen,

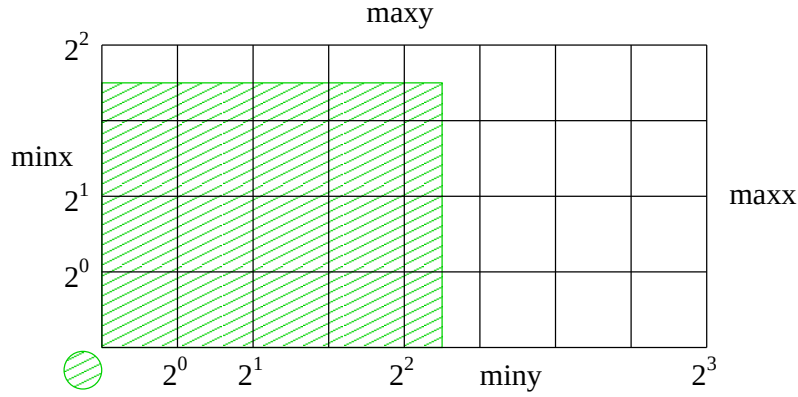


Abbildung 3.1: Dieses Gittersystem wird erzeugt, wenn der Benutzer einen Simulationsbereich und einen Interaktionsradius (grün) vorgibt.

ebenfalls „sinnvolle“ Zellindizes zugeordnet. Viele Aspekte der Simulation sind damit tatsächlich unabhängig von der Größe des Simulationsgebiets. Insbesondere funktionieren weiterhin alle Partikelinteraktionen, solange der euklidische Abstand gemessen wird. Es entsteht lediglich der Mehraufwand der Abstandsmessung zu Partikeln, die sich eine Kantenlänge des Simulationsgebiets $\pm r$ entfernt befinden, also ohne räumliche Nähe den selben Zellindex erhalten.

Auf diese Weise liegen zwar Partikel aus in x -Richtung benachbarten Zellen in der sortierten Liste nahe beieinander, nicht aber Partikel, die in y - oder z -Richtung in benachbarten Zellen liegen. In diesem Fall können tausende oder auch nur wenige Partikel zwischen dem betrachteten und dem benachbarten Partikel liegen. Um also alle Kollisionen auflösen zu können, werden Zellen benötigt, die genau diese Grenzen abdecken.

Abb. 3.2 zeigt die eindeutige Zuordnung jeder Position im Raum zu dem Gitter, in dem die zugehörige Zelle den Interaktionsbereich eines Partikels komplett umschreibt. Der Gitterindex ist die Summe der am Rand notierten Zahlen. Diese Zahlen werden beim Ermitteln der vier Zellindizes mittels einer zwei Bit feineren Zellindexbestimmung mit ermittelt und direkt als Gitterindex beim Partikel gespeichert.

Um nun alle Kollisionen auszuwerten, werden alle Flüssigkeitspartikel durchlaufen, jeweils die relevante Sortierung ausgelesen und in der gewählten sortierten Liste so viele nachfolgende und vorhergehende Partikel betrachtet, wie diese einen noch relevanten Zellindex besitzen. Da in x -Richtung zwei Bit mehr gespeichert werden, ist der Bereich, über den gesucht wird, nicht $4r$, sondern nur $3r$ (siehe Abb. 3.3). Nämlich die aktuelle x -Zelle, der Nachfolger und der Vorgänger.

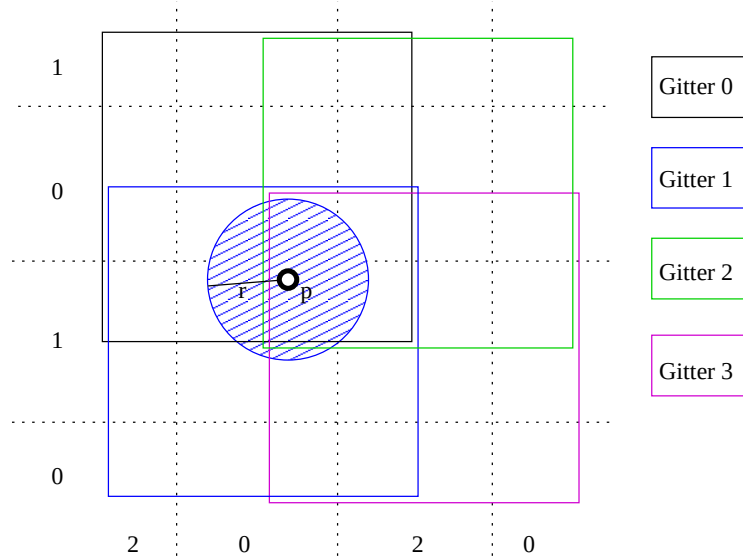


Abbildung 3.2: Dem Partikel zugeordnete Zellen in den vier Gittern mit Zuordnung des relevanten Gitters aus verfeinertem Gitterindex.

Hier ließe sich durch das Verwenden weiterer Bits eine noch feinere Unterteilung realisieren.

Bit	Größe	Verwendung bei
1	$4r$	y, z
2	$3r$	x
3	$2\frac{1}{2}r$	
4	$2\frac{1}{4}r$	
5	$2\frac{1}{8}r$	
...		

In meiner Anwendung ist eine Verwendung nicht benötigter Bits zu diesem Zweck nicht vorgesehen.

Da alle Kräfte symmetrisch sind, genügt es, bei beiden Kollisionspartnern die inversen Kräfte zu addieren. Um also nicht a mit b und b mit a kollidieren zu lassen, wird jetzt geprüft, ob der Index des Kollisionspartners größer ist als der des aktuellen Partikels. Ist dies der Fall, so wird die eigentliche Interaktion der Partikel berechnet. Um Kontrolle über das Verhalten der Flüssigkeit zu erlangen, verwende ich neben den SPH-Partikeln auch spezielle Partikel, die direkte Veränderungen an den SPH-Partikeleigenschaften bewirken. Diese haben immer einen höheren Index als SPH-Partikel. Näheres dazu in Kapitel 3.2.2 und 5.4.2.

Der Kollisionspartner kann also entweder selbst ein Flüssigkeitspartikel oder ein Kontrollpartikel sein.

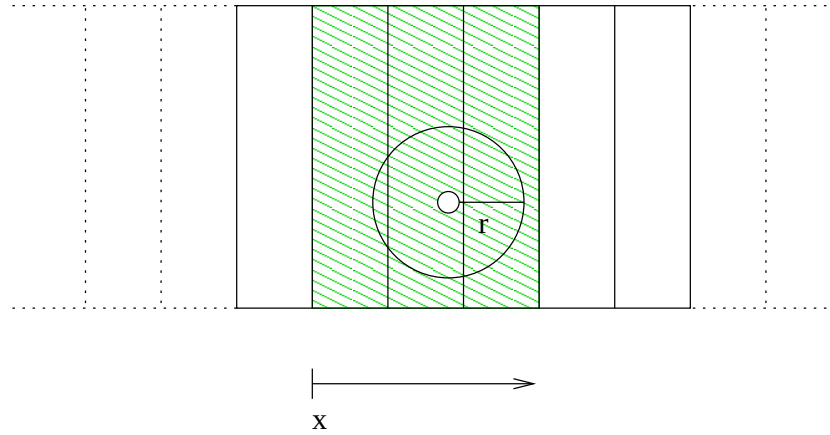


Abbildung 3.3: In x -Richtung wird eine feinere Zellunterteilung verwendet.

3.2.1 Flüssigkeitspartikel

Flüssigkeitspartikel kommen als Kollisionspartner in Frage, wenn der Abstand kleiner als r ist. Das heißt, es muss der Abstand zu dem wie eben beschrieben gefundenen Partikel gemessen werden. Ist dieser kleiner als r , so wird Flüssigkeitsphysik wie in Kapitel 2 beschrieben angewendet.

3.2.2 Kontrollpartikel

Um möglichst wenige Kontrollpartikel zu benötigen, wird hier die Tatsache ausgenutzt, dass nach

$$V_{\text{Zelle}}/\text{Kollisionsvolumen} = 4r * 4r * 3r / \left(\frac{4}{3}\pi r^3\right) = \frac{36}{\pi}$$

etwa 11.46 mal so viel Raum nach potentiellen Kollisionspartnern durchsucht wird, wie für die Physik überhaupt in Frage kommt. Bei der Kollision mit Kontrollpartikeln wird kein Partikelabstand gemessen, sondern der Abstand zur Kontrollpartikelebene (siehe Abb. 3.5). Ist dieser negativ, so tritt das Kontrollpartikel in Aktion. Wie in Abb. 3.4 dargestellt, wird der positive Abstand durch die Orientierung des Dreiecks bestimmt, das zur Initialisierung des Kontrollpartikels verwendet wurde.

Bemerkung: Solange Kontrollpartikel sich nicht bewegen, könnten sie auch einmal vorsortiert werden, um sie dann in einem Schritt eines Merge-Sort-Algorithmus mit der sortierten Liste der bewegten Partikel zu kombinieren. Dadurch wäre die Laufzeitordnung der Sortierung nun noch in der Anzahl der bewegten Partikel $n \log(n)$ und linear in der Anzahl der Kontrollpartikel.

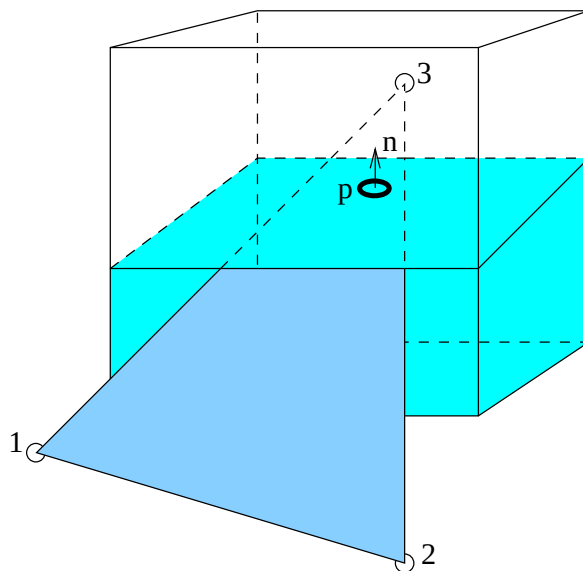


Abbildung 3.4: Die Orientierung des verwendeten Dreiecks bestimmt die Normalenrichtung.

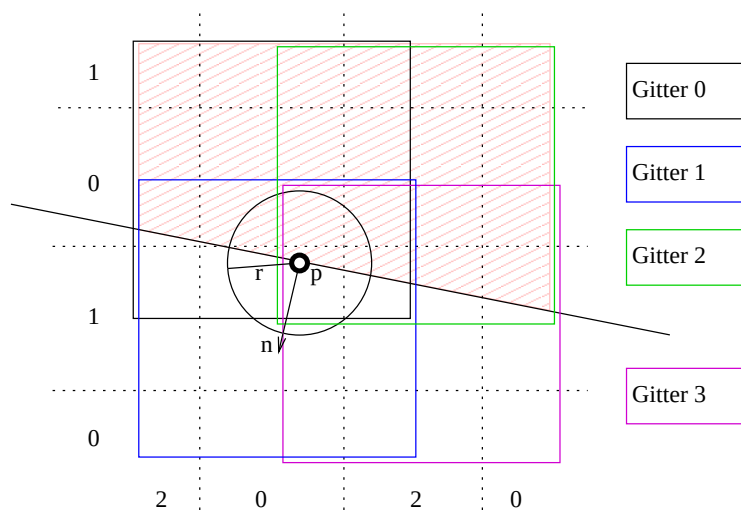


Abbildung 3.5: Lage des Kontrollpartikels in Verhältnis zum betroffenen Bereich.

Kapitel 4

Visualisierung

Zur Visualisierung wähle ich ausschließlich eine Art Marching-Cube Verfahren (MCA, siehe [3]). Anfangs ließ ich diesen Marching Cube (MC) auf der selben Gitterstruktur laufen wie die Kollisionsdetektion. Das erwies sich aber aus verschiedenen Gründen als nicht optimal. Die auf diese Weise konstruierte Hülle war zu eckig und die Bewegungen zu sprunghaft, da ja nur diskrete Gitterpositionen zur Verfügung standen. Mit den folgenden drei Ansätzen konnten einige Probleme behoben werden.

- Flächen höherer Ordnung
Da der MCA als kleinste Einheit Oktaeder erzeugt und diese durch einzelne herumfliegende Partikel sehr häufig zu Stande kommen, eignet er sich a priori nicht, auch nur annähernd runde Formen zu generieren. Diesem Problem konnte ich dank ATI-TRUFORM mit einer künstlichen Verfeinerung der Triangulierung mittels der Normaleninformation in jedem Vertex beikommen.
- Deformiertes Gitter
Um den Sprüngen von einer Gitterkoordinaten zur nächsten Herr zu werden, führte ich eine Verformung des MC-Gitters ein, die es insbesondere erlaubt, einzelne Tröpfchen darzustellen, ohne dass man eine Gitterstruktur erkennt.
- Verfeinertes Gitter
Meine Ansätze, das Gitter zu verfeinern, haben leider nicht den großen Erfolg gebracht, da die unangenehmsten Artefakte des MCA nicht beseitigt werden konnten.

4.1 Der Marching Cube Algorithmus

In seiner ursprünglichen Variante wird dieser Algorithmus zur Berechnung von Isoflächen skalarer Werte verwendet, die auf einem regelmäßigen Gitter bekannt sind. Im Dreidimensionalen werden alle „Würfel (Cubes) benachbarter Knoten“ (siehe Abb. 4.1) durchlaufen. Für jeden Würfel werden

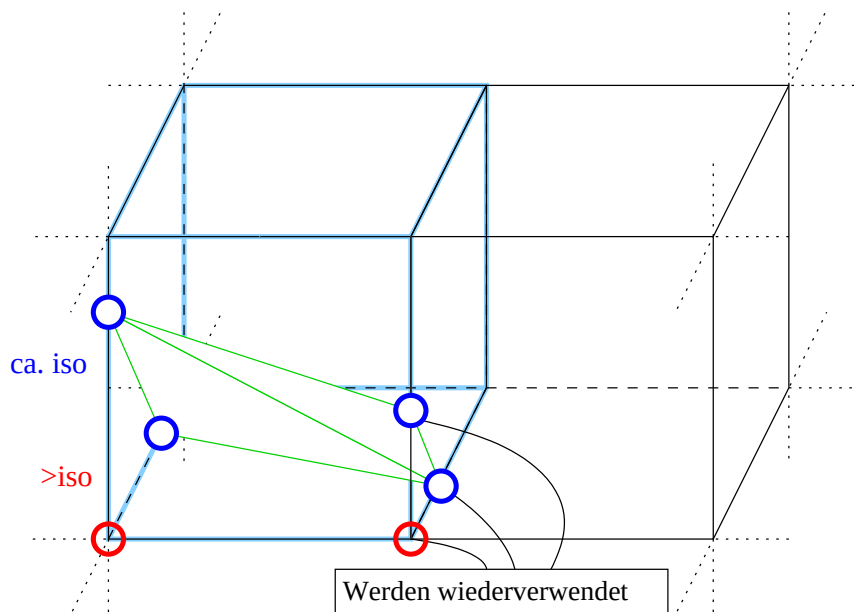


Abbildung 4.1: Marching Cube auf einem regelmäßigen Skalarfeld mit Knoten unterhalb eines Isowertes (rot), dadurch resultierenden Punkten auf den Kanten (blau) und der mit Hilfe einer Tabelle erzeugten zugehörigen Triangulierung (grün).

die Ecken bestimmt, die einen festen Isowert überschreiten. Auf Kanten, die die Isofläche durchdringen, da die beteiligten Ecken nicht beide ober- bzw. unterhalb des Isowertes liegen, wird mittels Interpolation bestimmt, an welcher Stelle der Isowert angenommen wird. Nun werden auf Grund der Eck-Konfiguration (einfacher Tabellenaufruf, siehe [3]) die ermittelten Punkte auf den Kanten zu einer Isofläche des einzelnen Würfels verbunden. Beim Durchlaufen des Skalarfeldes können die Daten des vorangegangenen Würfels, also Skalarwerte an schon ausgewerteten Ecken und Kanten wiederverwendet werden. So marschiert also der Würfel durch das Skalarfeld und erzeugt, unter der Annahme von Randpunkten des Skalarfeldes, die alle unterhalb des Isowertes liegen, eine geschlossene Hülle um alle Knoten, die oberhalb liegen. Die Vorschrift für die Triangulierung eines Würfels ist damit noch nicht eindeutig. Siehe dazu Abb. 4.2.

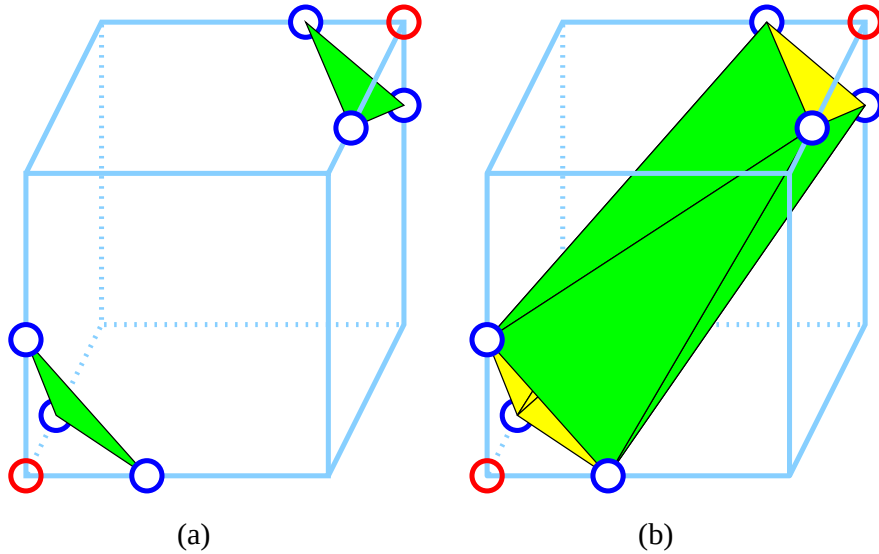


Abbildung 4.2: Wahlfreiheit der Triangulierung

Auf der Website [3] fand ich die Liste einer Beispielimplementierung des MCA. Diese habe ich, leicht abgewandelt, in meiner Arbeit verwendet. So stellte ich sicher, dass in Situationen wie in Abb. 4.2 stets Fall (a) gewählt wird, da ohnehin eines der störendsten Artefakte des Algorithmus *instabile*, durch Flackern auffallende Verbindungen zwischen einzelnen Tropfen oder Flüssigkeitsansammlungen sind.

Je näher sich Partikel kommen müssen, um als eine Zusammenhangskomponente dargestellt zu werden, desto größer ist auch die Wahrscheinlichkeit, dass die Physik eine parallele Bewegung auf einer kritischen Distanz verhindert, welche besonders zu derartigen Artefakten führen dürfte.

Bemerkung: Da auch sehr nah benachbarte Partikel diagonal benachbart sein können, könnte diese These auch falsch sein. Generell ist aber eine Bewegung nahe dem Grenzfall zwischen *verbunden* und *nicht verbunden* nie auszuschließen.

Die Physik ist zwar wie in Kapitel 3 gezeigt auf einem sehr großen Gebiet berechenbar. Nur leider liegt auf diese Weise nun auch kein für den MCA benötigtes Skalarfeld vor. Dieses auf dem gesamten Simulationsgebiet zu erzeugen würde wieder eingangs beschriebene Speichermengen verlangen.

4.2 A grid to march on

Um nun den MC loslaufen zu lassen, wird ein Gitter benötigt. Genau genommen benötigt der MCA aber nur jene Gitterzellen, deren Ecken sowohl innerhalb, als auch außerhalb der Flüssigkeit liegen. Ein guter Anfang sind also jene Zellen, die überhaupt Ecken in der Flüssigkeit haben.

4.2.1 Ecken bestimmen

Der erste Schritt ist also die Bestimmung dieser Ecken (siehe Abb. 4.4 auf Seite 23). Dazu verwendete ich eine Liste ähnlich der Listen, die für die Physik verwendet werden (siehe Kapitel 4.5). Nun wird der Zellindex aufeinanderfolgender Partikel verglichen und eine Liste angelegt, die jedem verwendeten Zellindex die Anzahl der darin befindlichen Partikel zuordnet. Auf diese Weise werden die Zellen der physikalischen Simulation zu den Ecken des MCA.

Bemerkung: Die hier ebenfalls vorhandene Dichte ρ der Partikel aufzusummieren hatte bei meinen Versuchen nur negative Effekte, sollte aber weiterhin in Betracht gezogen werden.

4.2.2 Gitterzellen zu den Ecken

Als nächstes wird eine Liste der an die gefundenen Ecken angrenzenden Würfel erzeugt. Da davon ausgegangen werden kann, dass der Zellindex der i -ten Ecke größer ist als der der $(i - 1)$ -ten, wurde der Würfel, der von der i -ten Ecke und

$$i\text{-te Ecke} + \text{Würfelgröße} \cdot \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

aufgespannt wird, noch nicht erzeugt und kann so *ungeprüft* als neuer Würfel in eine Würfelliste aufgenommen werden. Jeder andere an die i -te Ecke angrenzende Würfel könnte bereits als solch ein *ungeprüft aufgenommenener* Würfel in der Liste vorhanden sein.

Einschub: Alle acht an die Ecke grenzenden Würfel zu erzeugen und in eine Hashtable zu schreiben hätte zwar über 1000 Zeilen Code sparen können, wäre aber sicher wesentlich langsamer, da mein Code die spezielle Struktur des Problems ausnützt.

Deshalb wird nun auch eine Liste von Würfeln, „pending_cubes“, also „an-stehende Würfel“ habe ich sie getauft, durchlaufen, deren Zellindex im letzten Durchlauf noch als potentieller Nachbarwürfel eines noch kommenden neuen Würfels in Frage kam. Würfel, für die dies nicht mehr zutrifft, werden aus der pending_list gelöscht. Wird ein Nachbarwürfel gefunden, so wird die entsprechende Ecke des Nachbarwürfels als der aktuellen Ecke zugehörig markiert. Wird einer der acht Nachbarwürfel nicht gefunden, so wird dieser ebenfalls neu erzeugt. Alle neu erzeugten Würfel werden in die pending_list aufgenommen und es wird die nächste Ecke abgearbeitet.

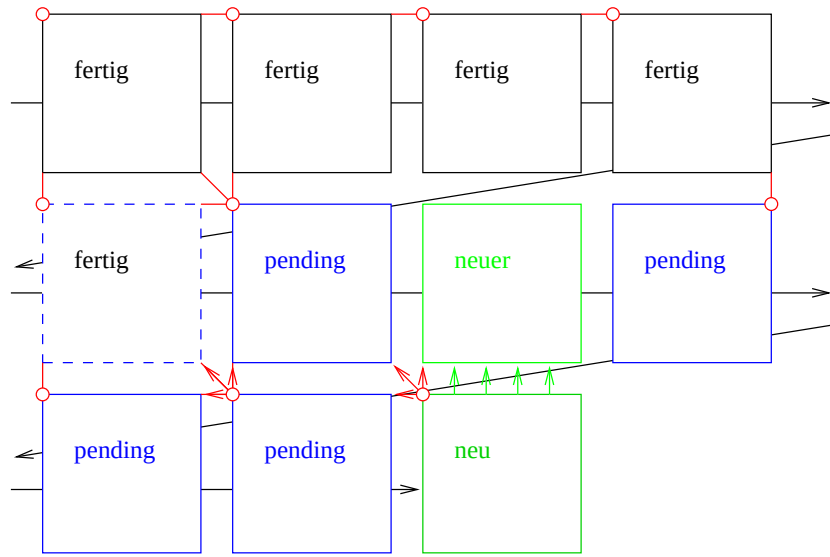


Abbildung 4.3: Bevor die Zelle „neu“ in die Liste der Zellen aufgenommen wird, werden die Zellen „pending“ auf Nachbarschaft geprüft und gemeinsame Punkte angepasst. Der benötigte Nachbar „neuer“ fehlt noch und wird deshalb mit „neu“ erzeugt und als „pending“ in die Zellliste aufgenommen.

Bemerkung: Da eine lange pending_list potentiell zu einer in der Zellanzahl quadratischen Laufzeitordnung führt, wäre es nötig, die Zellsortierung für den MCA an die räumliche Verteilung der Partikel zur Laufzeit anzupassen. Denkbar wäre hier etwa, die Koordinate, die die höchsten Bits im Zellindex liefert, auszutauschen, sobald im letzten Bild die Länge der pending_list mehr als $\beta\%$ der Eckliste enthielt.

Abb. 4.3 stellt zweidimensional die Entwicklung der Würfelliste dar. Der dunkelgrüne Würfel wird in diesem Fall erzeugt, weil seine obere linke Ecke in der zuvor erzeugten Eckliste stand, also „innerhalb der Flüssigkeit“ liegt. Beim Durchsuchen der pending_list (blau) werden der linke Nachbar (Zellindex - 1) und der obere linke Nachbar (Zellindex - Zeilenlänge - 1) gefunden

und der erste Würfel der zweiten Zeile (blau gestrichelt) aus der `pending_list` entfernt. Bei den zwei gefundenen Nachbarn wird an der entsprechenden Ecke (rote Pfeile) der selbe Verweis auf die aktuelle Ecke gespeichert wie bei dem erzeugten Würfel. Weil nach Durchlaufen der `pending_list` feststeht, dass der obere Nachbar (hellgrün) nie erzeugt wurde, wird er jetzt erzeugt und erhält die Eckinformation des aktuellen Würfels. Die beiden grünen Würfel werden der `pending_list` angefügt und es wird der nächste ECKeintrag gelesen.

4.2.3 Erzeugung der Oberflächenpunkte

Bezeichnung: Als Oberflächenpunkt bezeichne ich eine Ecke der gesuchten Oberflächentriangulierung. Als „Vorwärtsecke“ bezeichne ich die Ecke einer Zelle, die gemäß der Ecksortierung aus Kapitel 4.2.1 den höchsten Index hätte.

Die im letzten Abschnitt erzeugten Würfel würden bereits für einen MCA genügen. Um allerdings die Oberfläche wie in 4.4 beschrieben rund darstellen zu können, werden Normalen und damit auch weitere Informationen über die Umgebung der Oberflächenpunkte benötigt.¹

Um Nachbarschaften auszuwerten, wird die eben ermittelte Würfelliste sortiert und durchlaufen. Als erstes werden, falls benötigt, die Oberflächenpunkte der drei Kanten erzeugt, welche an die „Vorwärtsecke“ angrenzen. Sie bekommen die Koordinate derjenigen benachbarten Ecke, die innerhalb der Flüssigkeit liegt (nur diese liegt explizit vor), plus einen Offset, der proportional zur dritten Wurzel der für diese Ecke gespeicherten Partikelanzahl ist und bereits berechnet vorliegt (Volumen $\propto$ Partikelanzahl $\propto$ Kantenlänge³). Die Koordinaten in der Liste der Ecken liegen nicht auf den Ecken des zugrunde liegenden regelmäßigen Gitters. Siehe dazu Kapitel 4.3.

Mit dem Oberflächenvertex wird auch seine Normale in Abhängigkeit von der Eckkonfiguration des aktuellen Würfels erzeugt.

Exkurs: Auf welchen Kanten Oberflächenpunkte erzeugt werden müssen, hängt davon ab, welche Ecken innerhalb und welche außerhalb der Flüssigkeit liegen. Da diese Information in meinem Programm als 8-Bit-Zahl, nämlich ein Bit pro Ecke, vorliegt, überlegte ich mir, hier an Stelle der sechs `if`-Abfragen einen `switch` auf besagte 8-Bit-Zahl durchzuführen. Diesen Programmteil schrieb ich mit Hilfe der `if`-Abfragen automatisch. Daher sind diese knapp 4000 Zeilen Code auch in einer separaten Datei (siehe auch

¹Das Deaktivieren der Normalenbestimmung würde die Oberflächenberechnung deutlich beschleunigen, ist aber nicht implementiert.

5.2). Ist man nicht an Normalen interessiert, so sind von den 8 Bit nur 4 relevant und die Zeilenanzahl würde sich auf 90 reduzieren. Ich würde einen Performancegewinn erwarten, würde man genau diese Umwandlung vieler ifs in ein `switch` auch bei der Auswertung der MC-Tabelle anwenden.

Nun wird, wieder mittels einer `pending_list` (siehe Kapitel 4.2.2), die die Indizes der noch als Nachbarn in Frage kommenden Zellen hält, nach Nachbarn gesucht. Wird einer gefunden, so bekommt der aktuelle Würfel an den sich berührenden Kanten den Oberflächenpunktindex des Nachbarwürfels übertragen und die Normalen dieser Punkte werden mit der Eckkonfiguration des aktuellen Würfels angepasst. Auf diese Weise werden für jeden Oberflächenpunkt alle Ecken der 4 angrenzenden Würfel berücksichtigt.

Die an und für sich falsche Annahme eines regelmäßigen Gitters mit einer Dichte $\propto$ Partikelanzahl wird hier aus Performancegründen getroffen. Eine für korrekte Normalen benötigte Auswertung des auf den Partikeln gespeicherten Dichtefeldes findet nicht statt.

4.2.4 Verbinden der Oberflächenpunkte

In der Würfelloste ist nun also sowohl die Eckkonfiguration gespeichert als auch alle relevanten Daten der Oberflächenpunkte. Die Information, welche drei Punkte jeweils zu einem Dreieck verbunden werden, wird nun wie in Kapitel 4.1 aus einer Tabelle gelesen und als Triangulierungsliste gespeichert.

Die Triangulierungsliste ist mit der Liste der Oberflächenpunkte ausreichend, die MC-Oberfläche darzustellen. Um zwischen benachbarten Dreiecken auch mit Lichtberechnung glatte Übergänge zu erhalten ist die Liste der Normalen notwendig. Diese eignet sich aber auch, die Oberfläche runder zu machen. Siehe dazu Kapitel 4.4.

Abb. 4.4, 4.5 und 4.6 veranschaulichen nochmal in 2D die Vorgehensweise.

4.3 Deformiertes Gitter

Ein weiterer Ansatz, die Oberflächentriangulierung näher an die Partikel zu bringen, war es, beim Erzeugen der Ecken für den MCA auch die Schwerpunkte der Partikelpositionen an Stelle der Mittelpunktsskordinaten der Zellen zu speichern. Wie dann der MCA im Zweidimensionalen aussieht, veranschaulichen Abb. 4.7, 4.8 und 4.9. Hier wurden die selben Partikelpositionen verwendet wie bei Abb. 4.4, 4.5 und 4.6, S. 23ff.

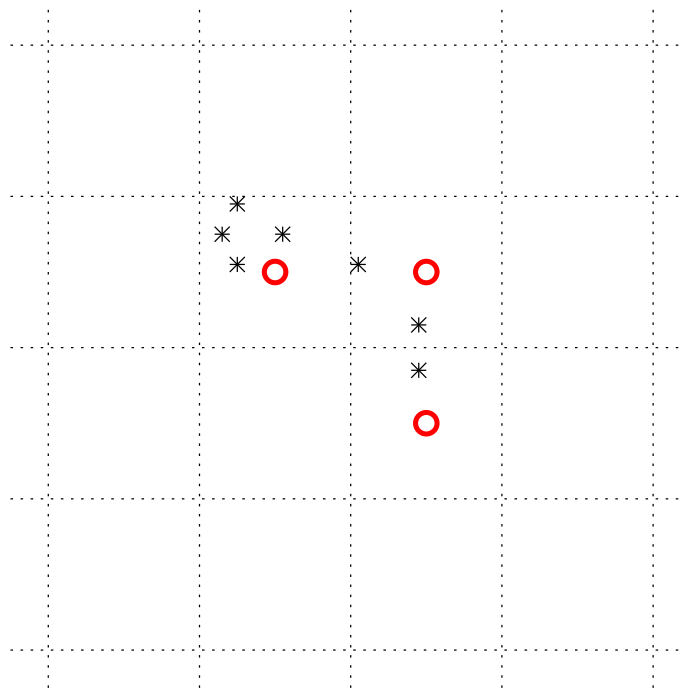


Abbildung 4.4: Die Anzahl der Partikel mit selbem Zellindex wird im Zellmittelpunkt als Ecke für den MCA gespeichert.

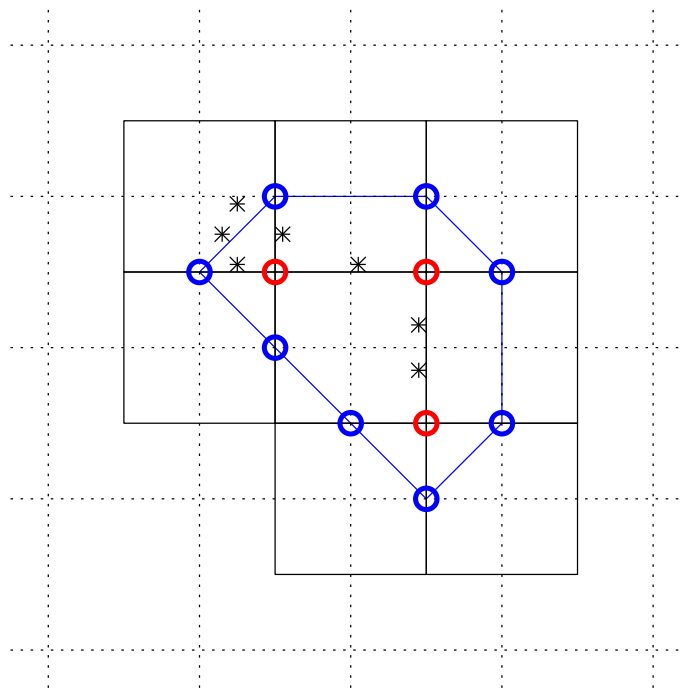


Abbildung 4.6: Punkte auf Kanten, die aus der Flüssigkeit herausführen, werden als Oberflächenpunkte markiert und verbunden.

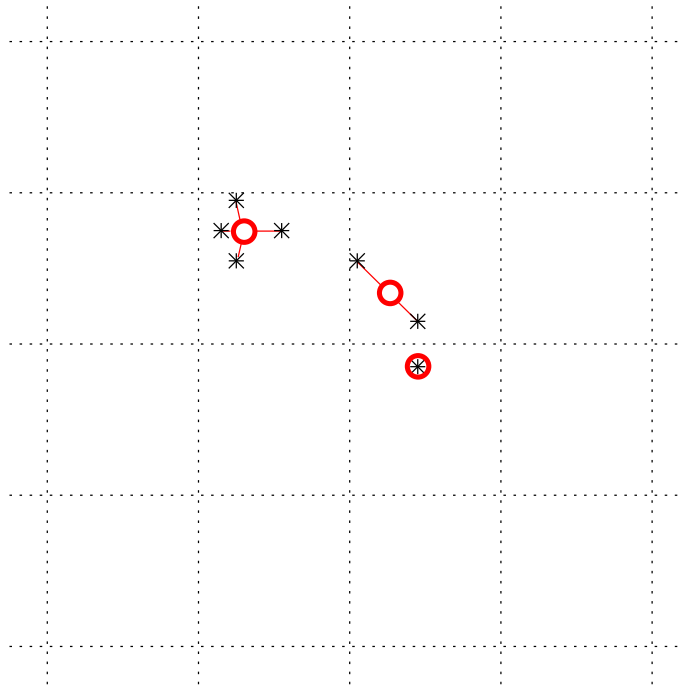


Abbildung 4.7: Schwerpunkt statt Zellmittelpunkt.

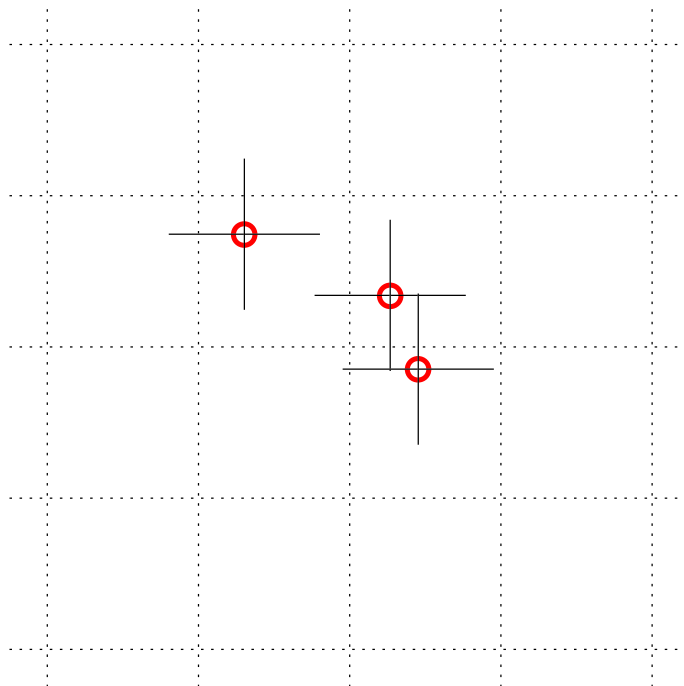


Abbildung 4.8: Die „Ecken“ des Marching Cube „Gitters“.

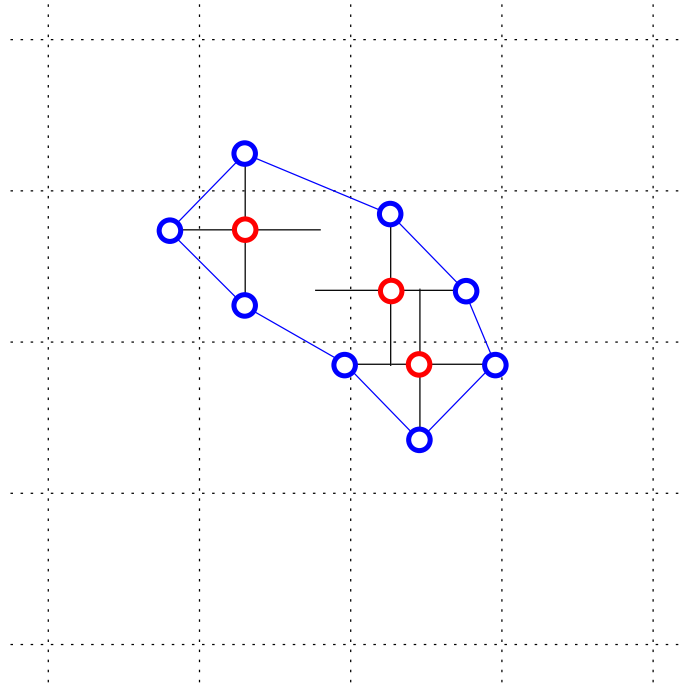


Abbildung 4.9: Eine natürlichere Hüllkurve.

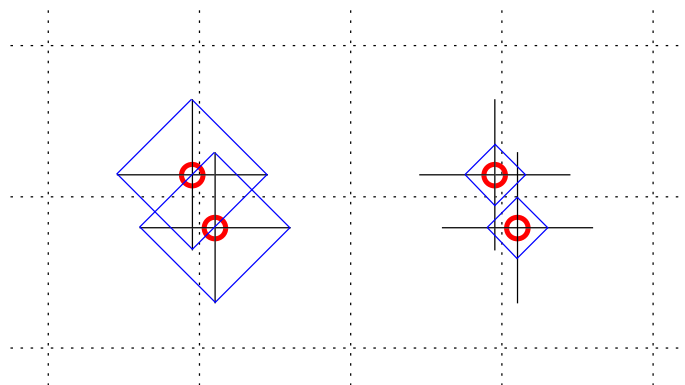


Abbildung 4.10: Verschieben des MC-Gitters kann zu Überschneidungen führen.

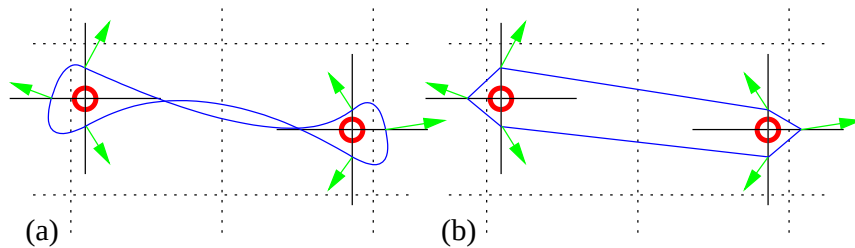


Abbildung 4.11: Entartete Flächen bei schlecht gewählten Normalen.

2. acht Würfel
3. sechs Oberflächenpunkte
4. acht Dreiecke

Wünschenswert wäre aber eine Kugel, was in meiner Anwendung nicht ganz erreicht wird. Ich suchte also einen Algorithmus, der Dreiecke anhand der Normaleninformation verfeinert, dabei aber jede Kante unabhängig von der dritten Ecke des Dreiecks unterteilt. Dies ist notwendig, will man die Verfeinerung ohne Nachbarschaftsinformationen durchführen. Dabei stieß ich auf ATIs TRUFORM-Erweiterung, die laut [4] bereits 2001 mit der Radeon 2 Grafikkarte auf Hardware implementiert war. Diese Erweiterung erzeugt Dreiecksgitter einer fest vorgegebenen Verfeinerungsstufe mit Gitterpunkten, die auf dem bikubischen Bezier-Patch liegen, das durch das Dreieck und seine Normalen definiert ist. Die Kanten von Bezier-Patches sind die zu den jeweiligen Ecken und ihren Normalen gehörenden Splinekurven. Damit hängt auch eine regelmäßige Unterteilung dieser Kanten nicht von der dritten Ecke ab und die TRUFORM-Flächen auf gleicher Stufe unterteilter benachbarter Dreiecke fügen sich nahtlos aneinander.

Die Normalen der so erzeugten Dreiecke können wahlweise linear oder kubisch interpoliert sein. Warum lineare Interpolation nicht so gute Resultate liefert, soll Abb. 4.12 veranschaulichen. Andererseits benötigt die kubische Interpolation mehr Rechenzeit.

In meiner Implementierung schalte ich grundsätzlich die Verfeinerung auf die maximale Stufe (7), da die Erzeugung der Oberflächendreiecke ohnehin weit mehr Zeit in Anspruch nimmt als die Unterteilung aller Dreiecke in 64 Teildreiecke durch eine ATI Radeon 9600. So läuft meine Simulation in manchen Fällen (wenig Physik) mit $5.000 * 64 = 320.000$ Dreiecken noch mit 30 Bildern pro Sekunde.

ATI TRUFORM™ zu verwenden, geht vorerst leider nur unter Windows. Dort erfordert es unter DIRECT X und unter Open GL lediglich das Einbinden der ATI-header und Einschalten der Erweiterung.

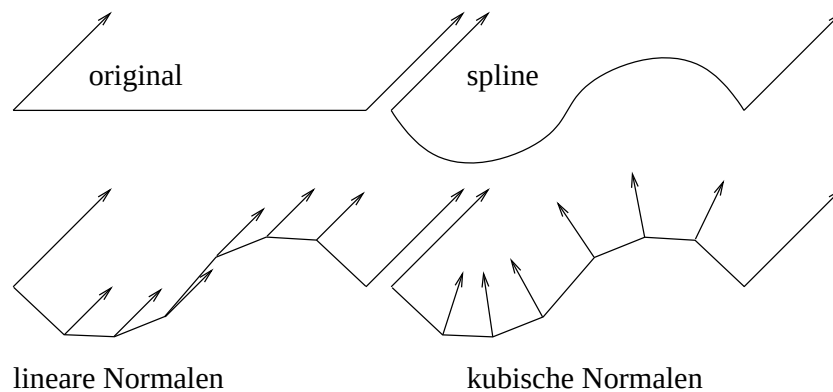


Abbildung 4.12: Linear vs. kubisch interpolierte Normalen bei ATIs TRUFORM-Erweiterung.

Bemerkung: Ein interessantes Thema ist die Frage, ob man ähnlich wie ATI TRUFORM Dreiecke auch mit unterschiedlichen Detailstufen effizient unterteilen kann. So könnte etwa für jede Dreiecksseite, die ja bei benachbarten Dreiecken übereinstimmen, abhängig vom Viewport eine Unterteilungsstufe festgelegt werden, um anschließend das Dreieck zu unterteilen.

4.5 Verfeinerung des Gitters

In den meisten Anwendungen führt ein verfeinertes Gitter für den MCA auch zu einer verbesserten Approximation der gesuchten Isofläche. Um zu testen, ob bei meinem Verfahren auch eine Verbesserung zu erzielen ist, führte ich zu den vier sortierten Listen der Partikel (siehe Kapitel 3) eine Fünfte ein, die nach einem Gitter mit halber Maschenweite sortiert war.

Da aber der ab Kapitel 4.2.1 beschriebene Ansatz jedes Partikel nur einer Ecke zuordnet und kein Höhenfeld um Partikel ausgewertet wird, zeigte die Verfeinerung des Gitters keine signifikante Verbesserung des Ergebnisses. Die Artefakte wurden zwar kleiner, aber wahrscheinlich auch häufiger. Eventuell hat die Verfeinerung den positiven Effekt, dass Partikel seltener verbunden dargestellt werden, solange ihr Abstand zueinander größer ist als der Interaktionsradius.

Kapitel 5

Anwendung

5.1 Systemvoraussetzungen

Die folgenden Voraussetzungen gelten für die Beispielimplementierung. Selbstverständlich kann zur Visualisierung auch jede andere Software verwendet werden, wodurch evtl. mehr RAM oder eine einfachere Grafikkarte benötigt würde. Die `Fluid` Klasse liefert Rohdaten, die auf verschiedenste Weise aufbereitet werden können.

- Betriebssystem
Die Quelldateien zu dieser Diplomarbeit wurden sowohl unter Windows-XP, als auch unter Linux (Debian & SuSE) compiliert. Da von ATI für Linux keine Treiber für deren TRUFORMs zur Verfügung standen, sind TRUFORMs unter Linux per Default deaktiviert.
- RAM
Je nach Partikelanzahl (Abhängig von der Fläche der Kontrollpolygone und dem Volumen des Fluids) können 15MB freier Speicher ausreichen. Ein Testlauf mit 380000 Partikeln ergab einen Speicherbedarf von unter 130MB.
- Grafikkarte
Zur Visualisierung empfiehlt sich eine ATI-Karte ab Radeon 9600, da diese TRUFORMs unterstützt. Leider existieren dafür aber wie gesagt nur Windows-Treiber.
- weitere Software
OpenGL, GLUT und ATIExtensions müssen für die Beispielimplementierung vorhanden sein.

5.2 Benötigte Dateien

- Fluid.h

Diese Datei enthält die Klasse `Fluid` mit den in (5.4) beschriebenen Methoden, die zur bloßen Anwendung meiner Ergebnisse ausreichend sind. Der Versuch, die Particleengine von der Physik und Visualisierung der Partikel als Flüssigkeit zu trennen gelang leider nicht, da mein Ansatz zur Visualisierung die Strukturen der Physik-Engine verwendet und damit alles in einer Klasse besser aufgehoben ist.

- autogenerated.h

Diese Datei resultierte aus dem Versuch, die Performance zu steigern, indem geschachtelte `ifs` durch ein `switch` ersetzt werden. Der Erfolg war zwar minimal, aber doch messbar. Somit findet sich in `Fluid.h` an entsprechender Stelle ein `#include "autogenerated.h"`.

- Particle.h

Diese Datei enthält die Klasse `Particle`. `Particles` speichern ihre Koordinate, Geschwindigkeit, Beschleunigung, Dichte, Informationen, ob sie Kontrollpartikel (siehe 3.2.2) sind, ihre Indizes in den vier sortierten Listen und in welcher der vier Listen sie komplett in einer Zelle Liegen.

- Vektor.h

Diese Datei enthält die Klasse `Vektor`. Eine Instanz von `Vektor` ist ein Vektor im dreidimensionalen euklidischen Vektorraum. Die Klasse unterstützt unter anderem Addition (`operator+(Vektor)`), Kreuzprodukt (`operator%(Vektor)`), Skalarprodukt (`operator*(Vektor)`), Vektorprodukt (`operator*(float)`, `operator/(float)`), Längennormierung (`norm(float)`) und Ermittlung der Länge (`abs()`).

5.3 Notation

Um Feldvariablen leserlich darzustellen, sind im Folgenden beispielsweise `float`-Werte zu Gruppen zusammengefasst dargestellt, die im Feld keine eigenen Untereinheiten darstellen. $\text{float} * \text{feld} = \{\vec{a}_1, \vec{a}_2, \vec{a}_3\}, \vec{a}_i = \{a_i^x, a_i^y\}$ heißt $\text{float} * \text{feld} = \{a_1^x, a_1^y, a_2^x, a_2^y, a_3^x, a_3^y\}$. `{}` steht hier übrigens nicht für die mathematische Mengenklammer, sondern stellt eine C++ Feldzuweisung dar.

5.4 Public member von Fluid

Die Fluidsimulation wurde in C++ realisiert und kann durch Verwendung der Klasse `Fluid` in Projekten verwendet werden. `Fluid` besitzt die folgenden öffentlichen member-Funktionen:

5.4.1 Konstruktor

Die Klasse `Fluid` enthält nur einen Konstruktor

```
Fluid::Fluid(const long maxparticlecount,  
const float particlesize,  
const float ax, const float ay, const float az,  
const float minx, const float miny, const float minz,  
const float maxx, const float maxy, const float maxz)
```

- `const long maxparticlecount`
Gibt die maximale Anzahl an zu verwendenden Partikeln an. Dies bezieht sich sowohl auf Kontrollpartikel, als auch auf die Fluid-Partikel. Ist `maxparticlecount` keine Zweierpotenz, so wird die nächst höhere Zweierpotenz verwendet. Ein Überschreiten dieser Limitierung im späteren Programmablauf führt zu einem Abbruch!
- `const float particlesize`
Gibt die Granularität an. Je kleiner die Partikel, desto mehr werden pro Volumen Flüssigkeit benötigt. Es werden auch mehr Kontrollpartikel benötigt.
- `const float ax, const float ay, const float az`
Eine globale Beschleunigung. In der Regel die Gravitation.
- `const float minx, const float miny, const float minz`
Minimale Koordinate des Simulationsbereichs.
- `const float maxx, const float maxy, const float maxz`
Maximale Koordinate des Simulationsbereichs. Wird vergrößert auf das nächstgrößere $\text{max} = \text{min} + 2^n * \text{cellsize} = \text{min} + 2^n * 4 * \text{particlesize}$

Auch wenn im hier verwendeten Verfahren die Kollisionsdetektion nicht mittels Expliziter Speicherung aller Zellen realisiert wird, ergibt sich doch eine Obergrenze bezüglich Partikelanzahl und Simulationsgebiet. Mit einer Zellgröße von `cellsize = 4 * particlesize` ergibt sich eine Unterteilung in

```
cellsx = (maxx - minx)/cellsize  
cellsy = (maxy - miny)/cellsize
```

$\text{cells}_z = (\text{maxz} - \text{minz}) / \text{cellsize}.$
 $\log_2(\text{maxparticlecount}) + \log_2(\text{cells}_x) + \log_2(\text{cells}_y) + \log_2(\text{cells}_z) + 3$
 muß ≤ 64 sein.

5.4.2 trianglelist2boundary

```
void Fluid::trianglelist2boundary(float * trianglelist,
unsigned int firsttriangleid, unsigned int lasttriangleid)
```

Mit dieser Funktion werden lokale Grenzflächen definiert. Die Grenzbedingung wirkt sich `particlesize` bis $\sqrt{41} \cdot \text{particlesize}$ über die Dreiecksränder hinaus aus, wodurch Dreiecke sich ungewollt überlappen können. Partikel, die sich von der *aktiven* Seite dem Dreieck nähern, werden auf die *erlaubte* Seite verschoben. Deshalb ist zu beachten, Wände mit genügend Abstand, also mindestens $6.5 \cdot \text{particlesize}$ anzuordnen.

Der Rechenaufwand ist nicht proportional zur Anzahl der Dreiecke, sondern zur Fläche der Dreiecke, da diese mit virtuellen Begrenzungspartikeln überzogen werden. Diese Begrenzungspartikel zählen zum `maxparticlecount`, sind aber bei weitem nicht so zeitraubend in der Berechnung, wie die bewegten Partikel, die die Flüssigkeit darstellen.

Nähere Informationen zu Kontrollpartikeln finden sich auch in Kapitel 3.2.2.

- `float * trianglelist`

Liste von Dreieckskoordinaten in der Form

$$\begin{aligned} *trianglelist &= \{Dreieck_0, Dreieck_1, \dots, Dreieck_n\} \\ Dreieck_i &= \{\vec{x}_i^0, \vec{x}_i^1, \vec{x}_i^2\} \\ \vec{x}_i^j &= \{x_i^j, y_i^j, z_i^j\} \\ 0 \leq i \leq n \quad \wedge \quad 0 \leq j \leq 2 \end{aligned}$$

Die *erlaubte* Seite ist die, in die $(\vec{x}_i^1 - \vec{x}_i^0) \times (\vec{x}_i^2 - \vec{x}_i^0)$ zeigt.

- `unsigned int firsttriangleid, unsigned int lasttriangleid`
 Die Liste von Dreieckskoordinaten wird nur von $Dreieck_{\text{firsttriangleid}}$ bis $Dreieck_{\text{lasttriangleid}}$ verwendet.
 $0 \leq \text{firsttriangleid} < \text{lasttriangleid} \leq n$

5.4.3 trianglelist2teleport

```
void Fluid::trianglelist2teleport(float target_x, float
target_y, float target_z, float * trianglelist, unsigned int
firsttriangleid, unsigned int lasttriangleid)
```

Wie bei `trianglelist2boundary` (5.4.2) wird auch bei `trianglelist2teleport` eine Grenzfläche definiert, bei deren Überschreiten eine Aktion ausgeführt wird. Die Partikel werden zu der Koordinate $(\text{target_x}, \text{target_y}, \text{target_z})^T$ „teleportiert“. Das heißt, sie werden an den Ort $(\text{target_x}, \text{target_y}, \text{target_z})^T + \vec{\delta}$ verschoben. $\vec{\delta}$ verhindert dabei, dass mehrere Partikel, die im gleichen Zeitschritt zum selben Ort teleportiert wurden exakt die selbe Koordinate erhalten. Generell ist aber zu beachten, daß ein Teleport evtl. viele Partikel an einen Ort zusammenführt und ist deshalb entweder mit einem `SetSpeed` (5.4.5) zu kombinieren, indem eine `SetSpeed`-Fläche an der Zielkoordinaten initialisiert wird, oder der Teleport wird durch ein `Shift` (5.4.4) ersetzt, welcher nicht zu Instabilitäten führt, solange im Zielgebiet kein starker Stau entsteht.

Nähere Informationen zu Kontrollpartikeln finden sich auch in Kapitel 3.2.2.

- `float target_x, float target_y, float target_z`
Die Koordinate, zu der die Flüssigkeit teleportiert werden soll.
- `float * trianglelist`
Liste von Dreieckskoordinaten wie bei `trianglelist2boundary` (5.4.2)
- `unsigned int firsttriangleid, unsigned int lasttriangleid`
Die Liste von Dreieckskoordinaten wird nur von $Dreieck_{\text{firsttriangleid}}$ bis $Dreieck_{\text{lasttriangleid}}$ verwendet.
 $0 \leq \text{firsttriangleid} < \text{lasttriangleid} \leq n$

5.4.4 `trianglelist2shift`

```
void Fluid::trianglelist2shift(float shift_x, float
shift_y, float shift_z, float * trianglelist, unsigned int
firsttriangleid, unsigned int lasttriangleid)
```

Ähnlich wie bei `trianglelist2teleport` (5.4.3) wird hier jedes Fluidpartikel, das mit einem der Begrenzungs-dreiecke kollidiert, d.h. in dessen aktiven Bereich gelangt, im Raum verschoben. Da das Ziel dieser Verschiebung aber eine echte Ausdehnung hat, und zwar die Selbe wie ihr Ursprung, kommt es hier nicht allein aus der Verschiebung zu Instabilität, solange die Flüssigkeit am Ziel zügig abfließen kann. Die Idee hinter dieser Funktion ist, dass Flüssigkeit, die sich im Innern eines Rohres befindet, nicht berechnet werden muß. Liegt das Ziel dieses Shifts allerdings in einem Bereich, wo die Flüssigkeit nicht von alleine abfließt, wird auch hier ein `SetSpeed` (5.4.5) benötigt, um zu vermeiden, daß Flüssigkeitspartikel an die Position anderer Partikel verschoben werden.

Nähere Informationen zu Kontrollpartikeln finden sich auch in Kapitel 3.2.2.

- `float shift_x, float shift_y, float shift_z`
(`shift_x, shift_y, shift_z`)^T ist der Vektor, um den die Flüssigkeit verschoben werden soll.
- `float * trianglelist`
Liste von Dreieckskoordinaten wie bei `trianglelist2boundary` (5.4.2)
- `unsigned int firsttriangleid, unsigned int lasttriangleid`
Die Liste von Dreieckskoordinaten wird nur von *Dreieck*_{firsttriangleid} bis *Dreieck*_{lasttriangleid} verwendet.
 $0 \leq \text{firsttriangleid} < \text{lasttriangleid} \leq n$

5.4.5 trianglelist2setspeed

```
void Fluid::trianglelist2setspeed(float speed_x, float  
speed_y, float speed_z, float * trianglelist, unsigned int  
firsttriangleid, unsigned int lasttriangleid)
```

Um an machen Stellen Instabilitäten zu vermeiden, ist es nötig, die Flüssigkeit künstlich zu bewegen. Dies ist zum Beispiel der Fall im Zielbereich von `trianglelist2teleport` (5.4.3). `trianglelist2setspeed` lässt sich aber auch dazu verwenden, an beliebigen Stellen im Simulationsgebiet Strömungen zu erzeugen.

Nähere Informationen zu Kontrollpartikeln finden sich auch in Kapitel 3.2.2.

- `float speed_x, float speed_y, float speed_z`
(`speed_x, speed_y, speed_z`)^T gibt die Geschwindigkeit für Partikel an, die auf die „aktive“ Seite der Begrenzungsfläche gelangen.
- `float * trianglelist`
Liste von Dreieckskoordinaten wie bei `trianglelist2boundary` (5.4.2).
- `unsigned int firsttriangleid, unsigned int lasttriangleid`
Die Liste von Dreieckskoordinaten wird nur von *Dreieck*_{firsttriangleid} bis *Dreieck*_{lasttriangleid} verwendet.
 $0 \leq \text{firsttriangleid} < \text{lasttriangleid} \leq n$

5.4.6 tetraederlist2liquid

```
void Fluid::tetraederlist2liquid(float * tetraederlist,  
unsigned int firsttetraid, unsigned int lasttetraid)
```

Mit dieser Funktion werden Fluidvolumina definiert. Die übergebenen Tetraeder werden mit Fluidpartikeln gefüllt. Dabei kann es zu einem Abbruch kommen, wenn der zuvor definiert Wert für `maxparticlecount` zu niedrig ist.

- `float * tetraederlist`

Liste von Tetraederkoordinaten in der Form

$$\begin{aligned} *tetraederlist &= \{Tetraeder_0, Tetraeder_1, \dots, Tetraeder_n\} \\ Tetraeder_i &= \{\vec{x}_i^0, \vec{x}_i^1, \vec{x}_i^2, \vec{x}_i^3\} \\ \vec{x}_i^j &= \{x_i^j, y_i^j, z_i^j\} \\ 0 \leq i \leq n \quad \wedge \quad 0 \leq j \leq 3 \end{aligned}$$

Die Reihenfolge der Tetraederecken spielt hier keine Rolle.

- `unsigned int firsttetraid, unsigned int lasttetraid`

Die Liste von Tetraederkoordinaten wird nur von $Tetraeder_{\text{firsttetraid}}$ bis $Tetraeder_{\text{lasttetraid}}$ verwendet.

5.4.7 progress

```
int Fluid::progress(const float dt)
```

Mit dieser Funktion wird der Zustand nach der übergebenen Zeit `dt` berechnet. Von `dt` hängt maßgeblich das Verhalten der Flüssigkeit ab. Bei zu großen Zeitschritten kommen numerische Instabilitäten zum Tragen. Außerdem können evtl. lokale Begrenzungen übersprungen werden. Der Rückgabewert gibt an, ob eine neue Position der Partikel berechnet wurde (1) oder nicht (0).

5.4.8 movingparticlecount

```
unsigned long Fluid::movingparticlecount(void)
```

Diese Funktion gibt die Anzahl aller Partikel zurück, die die Flüssigkeit darstellen.

5.4.9 boundaryparticlecount

```
unsigned long Fluid::boundaryparticlecount(void)
```

Diese Funktion gibt die Anzahl aller Partikel zurück, die für Kontrollmechanismen wie Grenzflächen (5.4.2), Teleporter (5.4.3), Shifter (5.4.4) und SetSpeed-Flächen (5.4.5) verwendet wurden.

5.4.10 particlecount

```
unsigned long Fluid::particlecount(void)
```

Diese Funktion gibt die Anzahl aller Partikel (`movingparticlecount()` + `boundaryparticlecount()`) zurück.

5.4.11 get_particlearray

```
void Fluid::get_particlearray(float ** VertexArray, unsigned  
long & arraylength)
```

Liefert die Positionen aller Partikel. Die ersten `movingparticlecount()` Koordinaten sind Flüssigkeitspartikel, die `boundaryparticlecount()` folgenden sind Koordinaten von Kontrollpartikeln.

- `float ** VertexArray`

Dies ist ein Zeiger auf ein Feld von Koordinaten der Form

$$\begin{aligned} *VertexArray &= \{\vec{x}_0, \vec{x}_1, \dots, \vec{x}_n\} \\ \vec{x}_i &= \{x_i, y_i, z_i\} \end{aligned}$$

verwendet den bereits reservierten Speicherbereich, falls `arraylength` ausreichend groß ist. Ansonsten wird der Speicherbereich freigegeben und ein neuer der Länge `3 * particlecount` reserviert.

- `unsigned long & arraylength`

Sollte mindestens `3 * particlecount` sein und wird wie eben beschrieben verwendet, um die Größe des Feldes gegebenenfalls anzupassen. Dieser Wert enthält nach Aufruf von `get_particlearray` *nicht* die Anzahl der geschriebenen Werte!

5.4.12 get_surfacegrid

```
int Fluid::get_surfacegrid(float ** VertexArrayPointer,  
float ** NormalArrayPointer, unsigned long &  
vn_arraylength, unsigned long & vn_count, unsigned long **  
triangleindicesPointer, unsigned long & ti_arraylength)
```

- `float ** VertexArrayPointer, float ** NormalArrayPointer,
unsigned long & vn_arraylength`

Die zwei `ArrayPointer` funktionieren mit `vn_arraylength` genauso, wie `VertexArray` und `arraylength` in der Methode `get_particlearray(float ** VertexArray, unsigned long`

& arraylength). `float ** VertexArrayPointer` ist dabei ein Zeiger auf das Feld der Ecken der Oberfläche der Flüssigkeit, `float ** NormalArrayPointer` ein Zeiger auf das Feld der zugehörigen Normalen.

$$\begin{aligned} *VertexArrayPointer &= \{\vec{v}_0, \vec{v}_1, \dots, \vec{v}_{vn_count-1}\} \\ \vec{v}_i &= \{v_i^x, v_i^y, v_i^z\} \\ *NormalArrayPointer &= \{\vec{n}_0, \vec{n}_1, \dots, \vec{n}_{vn_count-1}\} \\ \vec{n}_i &= \{n_i^x, n_i^y, n_i^z\} \end{aligned}$$

- `unsigned long & vn_count`
Gibt an, wieviele der `vn_arraylength` Feldelemente neu geschrieben und somit relevant sind.
- `unsigned long ** triangleindicesPointer`
Gibt an, welches Dreieck mit welchem Vertex inzidiert.

$$\begin{aligned} * \text{triangleindicesPointer} &= \{Dreieck_0, Dreieck_1, \dots, Dreieck_{n-1}\} \\ Dreieck_i &= \{\vec{x}_i^0, \vec{x}_i^1, \vec{x}_i^2\} \\ \vec{x}_i^j &= \{x_i^j, y_i^j, z_i^j\} \end{aligned}$$

n ist der Rückgabewert von `get_surfacegrid`

- `unsigned long & ti_arraylength`
Gibt an, wieviel Platz für `* triangleindicesPointer` reserviert wurde und wird gegebenenfalls erhöht.
- Rückgabewert `int get_surfacegrid`
Der Rückgabewert ist die Anzahl der erzeugten Dreiecke.

5.4.13 set_height_function

```
void Fluid::set_height_function(float (*height_function)(float x, float y))
```

Ermöglicht es, der Flüssigkeit ein auf dem ganzen Simulationsgebiet (global) gültiges Höhenfeld zu übergeben. Diese Funktion sollte mit geringstmöglichem Zeitaufwand auszuwerten sein, da jedes bewegte Partikel in jedem Zeitschritt gegen dieses Höhenfeld auf Gültigkeit geprüft wird. Im Einzelfall ist zu überlegen, ob lokale Begrenzungen mittels `trianglelist2boundary` (5.4.2) nicht performanter sind, als ein kompliziertes Höhenfeld.

- `float (*height_function)(float x, float y)`
Ist der Zeiger auf eine Funktion `float height_function(float, float)`.

5.4.14 `set_height_function_normal`

`void Fluid::set_height_function_normal(Vektor (*height_function_normal)(float x, float y))`

- `Vektor (*height_function_normal)(float x, float y)`
Ist der Zeiger auf die Funktion `Vektor height_function_normal(float, float)`. Diese sollte die nicht normierte Normale der Funktion, die `set_height_function` (5.4.13) übergeben wurde, zurückgeben. Falsche Normalen, z.B. mit negativer z -Komponente können zu unerwartetem Verhalten, bis hin zum Absturz der Software führen. Andererseits können mit Normalen, die nicht zur `set_height_function` übergebenen Funktion passen evtl. erwünschte Effekte oder Vereinfachungen erzielt werden.

5.4.15 `height_function`

`float Fluid::height_function(float x, float y)`

Gibt den Wert des Höhenfeldes bei (x, y) zurück.

5.4.16 `height_function_normal`

`Vektor Fluid::height_function_normal(float x, float y)`

Gibt die Normalenrichtung des globalen Höhenfeldes bei (x, y) zurück. Die Normale hat eine positive z -Komponente.

5.4.17 `minxyz`

`Vektor Fluid::minxyz(void)`

Gibt die minimale Ecke `Vektor( $min_x, min_y, min_z$ )` des Simulationsgebiets zurück.

5.4.18 maxxyz

`Vektor Fluid::maxxyz(void)`

Gibt die maximale Ecke `Vektor( $max_x, max_y, max_z$ )` des Simulationsgebiets zurück.

5.4.19 particleradius

`float Fluid::particleradius(void)`

Gibt den Partikelradius zurück. Das ist der maximale Abstand bei dem Partikel interagieren können. Kontrollpartikel wirken sich auf die gesamte Zelle in der sie sich befinden aus. Somit ist ihr Effekt nicht auf `particleradius` beschränkt, sondern geht deutlich darüber hinaus.

5.5 Die Beispielimplementierung

Mit Hilfe der auf der beiliegenden CD enthaltenen Visual Studio Projekt Datei bzw. für Linux dem ebenfalls enthaltenen Makefile lässt sich meine Beispielimplementierung kompilieren und ausführen.

5.5.1 Programmstart

Dieses Programm initialisiert eine Flüssigkeit als ein Tetraedervolumen, darunter einen Trichter aus Wanddreiecken, eine Shiftfläche am unteren Ende des Trichters welche die Flüssigkeit um die halbe Höhe zwischen Trichter und Nullpunkt transportiert, einen SetSpeed Bereich, der die Flüssigkeit in positiver x-Richtung bewegt wo die Flüssigkeit zuletzt noch auf eine Teleportfläche trifft, die sie wieder nach weit oberhalb des Trichters befördert.

Die Simulation startet im Pausemodus. Mausbewegungen rotieren die Szene um den Koordinatendreifuß im Nullpunkt, welcher in etwa die Mitte der Szene ist. Das Rechte-Maustaste-Menü ist selbsterklärend.

5.5.2 Tastaturbelegung

Mit Hilfe der Tastatur können einige Simulations- und Visualisierungsparameter verändert werden.

Taste	Effekt	Bemerkung
ESC	Ende	Beendet das Programm.
p	Pause	Schaltet die Physik, nicht aber die Oberflächenermittlung an/aus.
*, /	Geschwindigkeit	Verändern die Simulationsgeschwindigkeit um $\pm 10\%$.
+, -	Zoom	Ändern die Distanz zum Zentrum um 20%
c	Oberfläche	Schaltet die Berechnung/Darstellung der Flüssigkeitsoberfläche an/aus.
t	TRUFORM	Schaltet ATI TRUFORM an/aus. (nur bei Windows)
n	Normalen	Schaltet die Fluidnormalen an/aus falls die Flüssigkeitsoberfläche visualisiert wird, denn nur dann werden Normalen berechnet.
f	flat	Schaltet das Shading auf „flat“. Nurnoch eine Farbe pro Dreieck.
g	smooth	Schaltet das Shading auf „smooth“. Farbe des Dreiecks wird zwischen den Ecken interpoliert.
y	Volumen	Gibt das von der Flüssigkeitsoberfläche umhüllte Volumen auf die Konsole aus.
v	Partikel	Schaltet die Partikeldarstellung an/aus.
l	Licht	Schaltet die Beleuchtung an/aus.
e	filled	Schaltet die Vorderseiten auf „ausgefüllt“.
w	wireframe	Schaltet die Darstellung auf „nur Kanten“.
q	dots	Schaltet die Darstellung auf „nur Punkte“.
TAB		Schaltet die Rückseiten auf transparent.

Kapitel 6

Resultate

Wie schon an verschiedenen Stellen in dieser Diplomarbeit erwähnt, sind mit den vorgestellten Verfahren Partikelsimulationen mit minimalem Speicherbedarf möglich. Laut Windows Taskmanager benötigt die Simulation, aus der die folgenden Screenshots stammen, bei 830 dargestellten und 4000 für die Physik verwendeten Partikeln für die Physik und die Oberflächenberechnung gemeinsam unter 20MB. Dabei hängt der Speicherbedarf linear von der Partikelanzahl, der Anzahl der Oberflächenpunkte und der Anzahl der Zellen, die Flüssigkeit enthalten, ab.

Die Bildwiederholfrequenz liegt bei dieser Simulation auf einem Athlon 1.2GHz PC mit ATI Radeon 9600 zwischen 16 und 70 Bildern pro Sekunde.

Die Bilder in diesem Kapitel sind meist Stereoskopien, also Bilder der selben Szene aus leicht variierten Blickwinkeln. Betrachtet man diese Bilder, indem man versucht, hindurch zu blicken, so erfasst das linke Auge das linke Bild und das rechte Auge das rechte Bild und es entsteht ein dreidimensionaler Eindruck. Um das Fokussieren zu erleichtern, ist eine möglichst helle Beleuchtung hilfreich.

Auf diese Weise kann schon bei der Punktwolke in Bild 6.1 die räumliche Verteilung dieser Punkte erfasst werden. Diese Punkte sind die Zentren der Partikel, auf denen die Physik simuliert wird.

In Bild 6.2 wird die selbe Punktverteilung, allerdings mit den Verfahren aus Kapitel 4, dargestellt. Die Oberfläche ist rund und wirkt organisch.

Ohne Abrunden der Oberfläche sieht man allerdings sofort, dass das Gitter des Marching Cube Algorithmus sehr grob ist (siehe Abb. 6.3). Tropfen erscheinen als Oktaeder, die Lichtquellen erzeugen keine Spiegellinien.

Dass die wie in Abb. 4.10 (a) auf Seite 27 dargestellte Überschneidung von Flächen auch tatsächlich beobachtet werden kann, zeigt Bild 6.4. Entartungen, wie in Bild 4.11 (a) beschrieben, sind mir seit der Verfeinerung des



Abbildung 6.1: Partikel einer Flüssigkeit, die auf eine Ebene strömt.

Gitters nicht mehr aufgefallen.

In Abb. 6.5 verteilen sich Partikel auf einem großen Gebiet. Gerade in Szenen, in denen nicht viele Partikel interagieren, ist die Bildrate besonders hoch. Auch wenn hier ca. 5.000¹ Dreiecke die Oberfläche darstellen, werden noch über 35 Bilder pro Sekunde erreicht. Die Oberflächenberechnung wird nur dann langsam, wenn sich viel Oberfläche auf der selben z -Koordinate befindet.

¹5.000 mal 64 wegen der ATF-TRUFORMs.

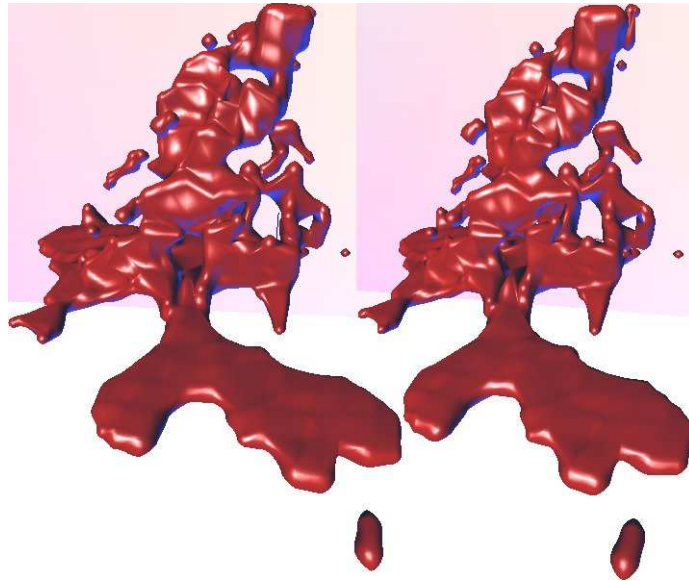


Abbildung 6.2: Die Oberflächendarstellung zur Partikelverteilung aus Abb. 6.1.

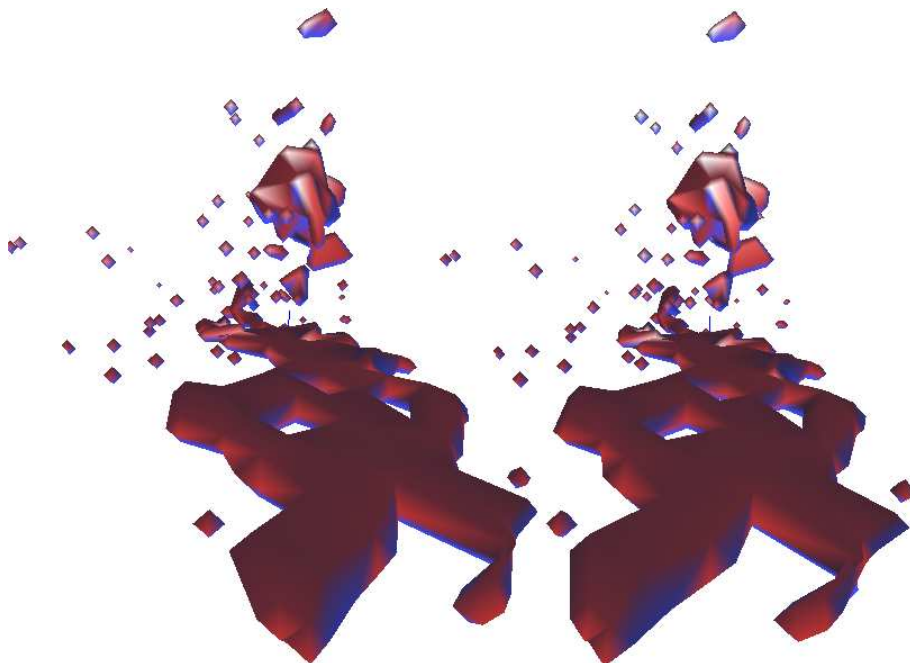


Abbildung 6.3: Ohne ATI-TRUFORM wirkt alles eckig und man sieht keine Spiegellinien.

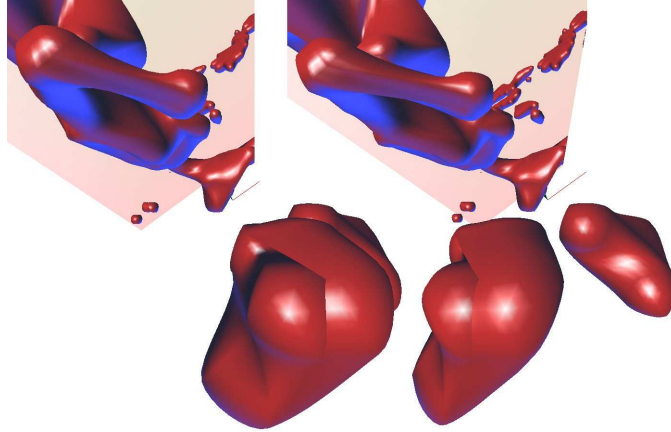


Abbildung 6.4: Selbstüberschneidung der Oberfläche.

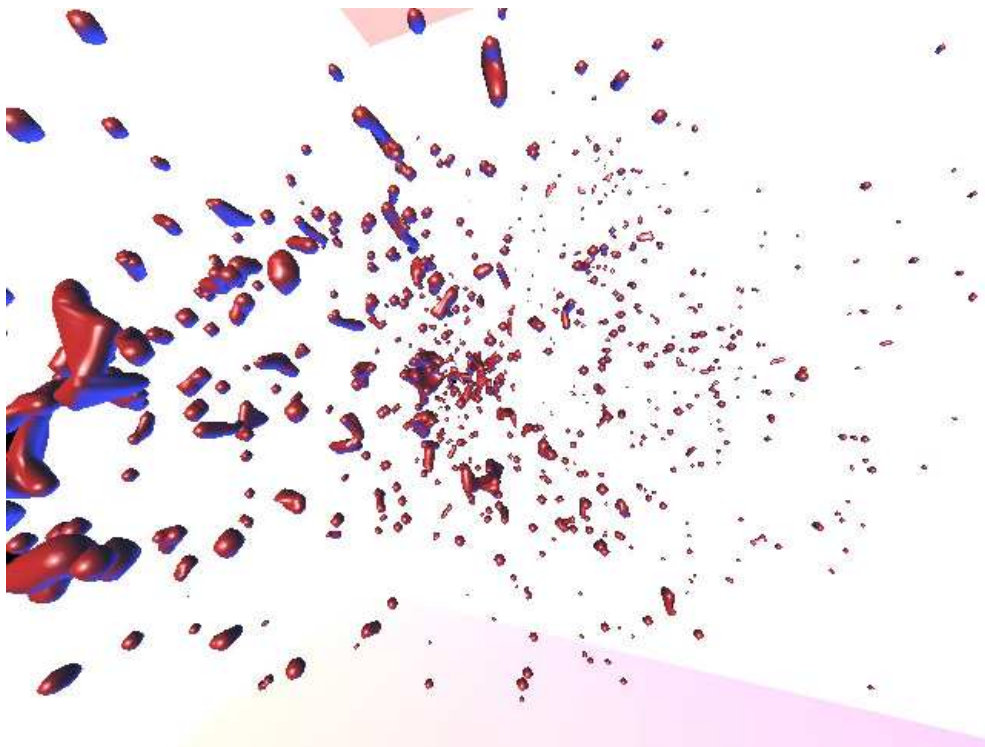


Abbildung 6.5: Spritzeffekt.

Kapitel 7

Ausblick

Auch wenn die hier gezeigten Resultate noch nicht nach Wasser aussehen, denke ich, dass schon bald erste Anwendungen entwickelt werden, die vielleicht nur kleine Mengen Wasser, oder zumindest nicht-transparente Flüssigkeiten, in interaktiven Umgebungen integriert zeigen werden.

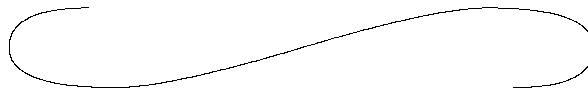
Obwohl das Trennen und Vereinen von Tropfen mit meinem Marching-Cube-Algorithmus unschöne Artefakte erzeugt, kann er eventuell in modifizierter Form, insbesondere mit einem besseren Algorithmus zur Bestimmung der Normalen als schnelle Methode zur Erzeugung runder Hüllflächen weiter Anwendung finden.

Veränderungen, die nötig wären, um Flüssigkeiten verschiedener physikalischer Eigenschaften zu simulieren stehen in keinem Widerspruch zu den vorgestellten Methoden. Im Gegenteil. So wie die Kontrollpartikel nur in der Physik von den Flüssigkeitspartikeln unterschieden werden, könnten auch, theoretisch beliebig viele, verschiedene Flüssigkeiten miteinander interagieren.

Kapitel 8

Danksagung

Abschließend möchte ich mich bei den Mitarbeitern des Lehrstuhls für Computer Graphik & Visualisierung der TU München, insbesondere Herr Prof. Dr. Westermann, sowie bei Prof. Dr. Huckle bedanken, dass sie meine Diplomarbeit in dieser Form möglich gemacht haben. Aber auch Freunde und Familie die mir mit Rat und Tat zur Seite standen möchte ich danken.



Literaturverzeichnis

- [1] Nick Foster & Ronald Fedkiw:
„*Practical animation of liquids*“,
Proceedings of ACM SIGGRAPH 2001.
- [2] Matthias Müller, David Charypar & Markus Gross:
„*Particle-Based Fluid Simulation for Interactive Applications*“,
Department Institute of Technology Zürich (ETHZ), Schweiz, 2003.
- [3] Paul Bourke:
„*Polygonising a scalar field*“,
<http://astronomy.swin.edu.au/pbourke/modelling/polygonise/>, 2003.
- [4] <http://www.ati.com>
- [5] <http://scienceworld.wolfram.com/physics/Navier-StokesEquations.html>
- [6] Simon Premože, Tolga Tasdizen, James Bigler, Aaron Lefohn & Ross T. Whitaker:
„*Particle-Based Simulation of Fluids*“,
University of Utah, 2003.
- [7] Olaf Kessel-Deynet:
„*Berücksichtigung ionisierender Strahlung im Smoothed-Particle-Hydrodynamics-Verfahren und Anwendung auf die Dynamik von Wolkenkernen im Strahlungsfeld massiver Sterne*“,
Ruprecht-Karls-Universität Heidelberg, 1999.